LAPORAN TUGAS BESAR
PROYEK BASIS DATA LANJUT

Disusun Oleh:

1. Tri Haiji Januarli (G1A024069)
2. R. Noor Fikhri (G1A024075)
3. Ikhsan Rihan Farli (G1A024083)

Nama Asisten Dosen :

1. Azzahra Faranisa (G1A023010)
2. Diosi Putri Arlita (G1A023012)
3. Salla Fikriyatul Arifah (G1A023015)
4. Fidia Dewi Wulandari B (G1A023040)
5. Anisa Julianti (G1A023052)
6. Ghazi Al-Ghifari (G1A023053)
7. Ayu Dewanti Suci (G1A023057)
8. Dinda Krisnauli Pakpahan (G1A023076)
9. Khaylillah Shafaraly Irnanda (G1A023079)
10. Muhammad Ryan Al Habsy (G1A023093)

Dosen Pengampu :

1. Arie Vatresia, S.T., M.T., Ph.D., IPP
2. Dr. Etis Sunandi, S.Si., M.Si

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS BENGKULU

2026

KATA PENGANTAR

Puji syukur kami panjatkan kehadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, kami dapat menyelesaikan laporan praktikum ini dengan baik. Laporan ini disusun sebagai bagian dari tugas mata kuliah Basis Data dengan tema Sistem Informasi Perpustakaan Digital (DigiLib) Berbasis Web. Melalui penyusunan laporan ini, kami mempelajari bagaimana konsep basis data relasional diterapkan dalam manajemen literatur, khususnya dalam pengelolaan data buku, keanggotaan, dan transaksi sirkulasi secara digital. Sistem DigiLib ini diharapkan mampu membantu pihak perpustakaan dalam mengelola layanan secara lebih efektif, efisien, dan transparan bagi seluruh civitas akademika.

Kami mengucapkan terima kasih yang sebesar-besarnya kepada Bapak Arie Vatresia, S.T., M.T., Ph.D., IPP dan Bapak Dr. Etis Sunandi, S.Si., M.Si selaku dosen pengampu mata kuliah, serta kepada asisten dosen praktikum yang telah membimbing kami selama proses pembelajaran dan penyusunan proyek ini. Semoga laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembaca dalam memahami penerapan sistem informasi berbasis web dan basis data di lingkungan institusi pendidikan.

Akhir kata, kami menyadari bahwa laporan ini masih memiliki keterbatasan. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi perbaikan di masa mendatang.

Bengkulu, 23 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
DAFTAR GAMBAR iii
BAB I LANDASAN TEORI 1
BAB II SOAL DAN PEMBAHASAN 3
BAB III PENUTUP 58
A. Kesimpulan 58
B. Saran 58
DAFTAR PUSTAKA 59
JOBDESK 60

DAFTAR GAMBAR

Gambar 2.1 ERD SI Perpustakaan Digital 3
Gambar 2.2 Source Code Koneksi Database (koneksi.php) 4
Gambar 2.3 Source Code Pengujian Koneksi Database (cek_tabel.php) 6
Gambar 2.4 Source Code Autentikasi Login (api_login.php) 7
Gambar 2.5 Tampilan Form Input Login 9
Gambar 2.6 Tampilan Autentikasi Berhasil 9
Gambar 2.7 Output Autentikasi Gagal (Akses Ditolak) 9
Gambar 2.8 Tampilan Pop-up Bantuan Login 10
Gambar 2.9 Source Code Pendaftaran Akun Baru (api_register.php) 11
Gambar 2.10 Antarmuka Formulir Daftar Akun Baru 13
Gambar 2.11 Source Code Membuat Tabel Users 13
Gambar 2.12 Tambah Data Users 14
Gambar 2.13 Tabel Users 15
Gambar 2.14 Source Code Logika Dashboard Admin (api_admin_dashboard.php) 16
Gambar 2.15 Tampilan Halaman Beranda (Sisi Admin) 18
Gambar 2.16 Source Code Logika Pengambilan Data Anggota (api_admin_users.php) 19
Gambar 2.17 Antarmuka Keanggotaan (Sisi Admin) 20
Gambar 2.18 Source Code Membuat Tabel Buku 21
Gambar 2.19 Tambah Data Buku 22
Gambar 2.20 Tabel Buku 23
Gambar 2.21 Output Update Buku Dengan Id = 3 24
Gambar 2.22 Source Code Logika Pengambilan Katalog Buku (api_buku.php) 25
Gambar 2.23 Antarmuka Katalog Buku Terintegrasi (Sisi Anggota) 27
Gambar 2.24 Source Code Logika Tambah Koleksi Buku (api_tambah_buku.php) 27
Gambar 2.25 Source Code Logika Update Buku (api_admin_buku) 29
Gambar 2.26 Source Code Update Koleksi Buku (api_edit_buku.php) 32
Gambar 2.27 Source Code Menghapus Koleksi Buku (api_delete_buku.php) 34
Gambar 2.28 Antarmuka Input Buku (Sisi Admin) 36
Gambar 2.29 Antarmuka Input Buku (Sisi Admin) 36
Gambar 2.30 Verifikasi Penghapusan pada Tabel Buku (Sisi Admin) 36
Gambar 2.31 Edit Buku (Sisi Admin) 37
Gambar 2.32 Notifikasi Keberhasilan Pengeditan Buku (Sisi Admin) 37
Gambar 2.33 Membuat Tabel Peminjaman 38

Gambar 2.34 Tambah Data Peminjaman 39
Gambar 2.35 Tabel Peminjaman 40
Gambar 2.36 Source Code Logika Peminjaman Buku (api_pinjam.php) 41
Gambar 2.37 Antarmuka Formulir Peminjaman Buku (Sisi User) 43
Gambar 2.38 Source Code Logika Perpanjangan Peminjaman (api_perpanjang.php) 44
Gambar 2.39 Antarmuka Konfirmasi Perpanjangan Masa Pinjam 46
Gambar 2.40 Source Code Logika Relasi Data Sirkulasi (api_admin_peminjaman.php) 46
Gambar 2.41 Antarmuka Katalog Koleksi Buku (Sisi Admin) 48
Gambar 2.42 Source Code Logika Pengembalian Buku (api_kembali_buku.php) 49
Gambar 2.43 Konfirmasi Pengermbalian Buku (Sisi Admin) 51
Gambar 2.44 Source Code Logika Riwayat Peminjaman (api_user_data.php) 52
Gambar 2.45 Antarmuka Rak Buku Virtual dan Riwayat Peminjaman (Sisi User) 53
Gambar 2.46 Membuat Tabel Ulasan 54
Gambar 2.47 Tabel Ulasan 55
Gambar 2.48 Source Code Pemberian Ulasan Buku (api_rating.php) 55
Gambar 2.49 Antarmuka Formulir Pemberian Ulasan (Sisi User) 57
Lampiran 1 60
Lampiran 2 60

BAB I
LANDASAN TEORI

Basis Data Basis data (*database*) merupakan fondasi utama dalam arsitektur sistem informasi modern. Secara terminologi, basis data adalah sekumpulan data yang terorganisir secara logis dan saling berelasi, yang disimpan secara digital dalam media penyimpanan komputer. Basis data berfungsi untuk mengeliminasi redundansi data, menjaga integritas informasi, serta memungkinkan akses data yang cepat dan akurat. Dalam pengembangan aplikasi, basis data tidak hanya sekadar tempat penyimpanan, tetapi juga sistem yang mengatur bagaimana data didefinisikan, dikonstruksi, dan dimanipulasi. Pohan (2002) dalam Isa & Thomas (2022) menekankan bahwa penggunaan *Structured Query Language* (SQL) pada basis data relasional menjadi standar utama karena kemampuannya dalam menangani relasi antar tabel yang kompleks, yang sangat dibutuhkan dalam mengelola entitas yang saling terhubung seperti data anggota, buku, dan transaksi sirkulasi.

Perpustakaan digital merupakan bentuk transformasi fundamental dari metode pengelolaan koleksi literatur konvensional ke dalam format elektronik yang terintegrasi. Menurut Fajri (2022), perpustakaan digital adalah penerapan teknologi informasi sebagai sarana untuk menyimpan, mendapatkan, dan menyebarluaskan informasi ilmu pengetahuan secara masif dalam format digital. Hal ini memungkinkan pengguna (*user*) untuk mengakses sumber-sumber informasi dari mana saja tanpa terikat oleh batasan fisik gedung perpustakaan. Implementasi sistem ini bertujuan untuk meminimalisir kendala operasional seperti proses pendaftaran dan sirkulasi buku yang seringkali masih dilakukan secara manual menggunakan buku besar, yang rentan terhadap kehilangan data dan inefisiensi waktu (Isa & Thomas, 2022). Dengan adanya sistem informasi, seluruh data bibliografi dapat dikelola secara terstruktur, memberikan keamanan data yang lebih terjamin, serta memfasilitasi pengambilan keputusan manajerial yang lebih akurat.

Pengembangan antarmuka pengguna (*User Interface*) pada sistem perpustakaan digital memerlukan sinergi antara struktur, estetika, dan fungsionalitas. Lukmana et al. (2023) menekankan bahwa penggunaan HTML (*Hypertext Markup Language*) berfungsi sebagai kerangka dasar untuk mendefinisikan elemen-elemen web seperti tabel data buku, formulir input anggota, dan tata letak katalog. CSS (*Cascading Style Sheets*) kemudian berperan krusial dalam menciptakan desain yang responsif melalui teknik *grid* atau *flexbox*, memastikan aplikasi tetap fungsional saat diakses melalui perangkat *mobile* maupun desktop. Untuk meningkatkan pengalaman pengguna (*User Experience*), JavaScript digunakan untuk memberikan elemen interaktif seperti validasi data di sisi klien, fitur

pencarian *real-time* (AJAX), dan filter kategori buku tanpa perlu melakukan pemuatan ulang halaman (*page refresh*). Kombinasi ini bertujuan menciptakan antarmuka yang intuitif sehingga anggota perpustakaan dapat mengeksplorasi koleksi digital dengan hambatan teknis minimal.

Dalam pengembangan sistem berbasis web, PHP memegang peranan vital sebagai bahasa pemrograman *server-side* yang mengelola logika bisnis aplikasi. Sofwan (2019) menjelaskan bahwa PHP memiliki kemampuan unggul dalam memproses data dinamis dan berkomunikasi langsung dengan basis data. Dalam konteks perpustakaan, PHP bertanggung jawab untuk menangani proses autentikasi (login admin/anggota), menghitung masa peminjaman, hingga mengelola status ketersediaan stok buku secara otomatis. Keunggulan PHP terletak pada kemampuannya untuk disisipkan langsung ke dalam kode HTML, yang memungkinkan pengembang membangun halaman katalog yang kontennya berubah-ubah sesuai dengan isi database secara *real-time***.**

Keberhasilan sebuah sistem informasi sangat bergantung pada kualitas perancangan basis datanya. MySQL digunakan sebagai *Database Management System* (DBMS) karena keandalannya dalam menangani volume data yang besar dengan kecepatan akses yang tinggi. Subekti dan Pratama (2024) menyoroti pentingnya struktur database relasional untuk menyimpan informasi koleksi, data pengguna, serta riwayat transaksi peminjaman. Melalui penggunaan *Foreign Key*, tabel-tabel seperti buku, kategori, users, dan peminjaman dapat saling terintegrasi secara logis.

Untuk menghasilkan perangkat lunak yang stabil, metodologi pengembangan yang sistematis seperti *Waterfall* seringkali menjadi pilihan utama. Lukmana et al. (2023) menjelaskan bahwa metode ini memberikan alur kerja yang jelas, mulai dari analisis kebutuhan pengguna, perancangan sistem menggunakan UML (*Unified Modeling Language*), tahap pengkodean, hingga pengujian. Pemodelan dengan UML, seperti *Use Case Diagram* dan *Activity Diagram*, membantu pengembang memetakan alur sirkulasi buku dengan jelas. Tahap akhir yang tidak kalah penting adalah pengujian menggunakan metode *Black Box Testing*, di mana sistem diuji untuk memastikan bahwa setiap fitur : seperti modul pendaftaran, pencarian, dan pengembalian buku, berjalan sesuai dengan fungsi yang diharapkan tanpa kegagalan logika.

Sistem informasi perpustakaan digital harus dirancang dengan memperhatikan aspek skalabilitas, yaitu kemampuan sistem untuk tumbuh seiring bertambahnya jumlah koleksi dan pengguna. Laporan World Bank Group (2016) mengenai Sistem Informasi Manajemen menekankan bahwa sistem yang efektif adalah sistem yang mampu menghasilkan data berkualitas tinggi untuk digunakan dalam perencanaan jangka panjang.

BAB II

SOAL DAN PEMBAHASAN

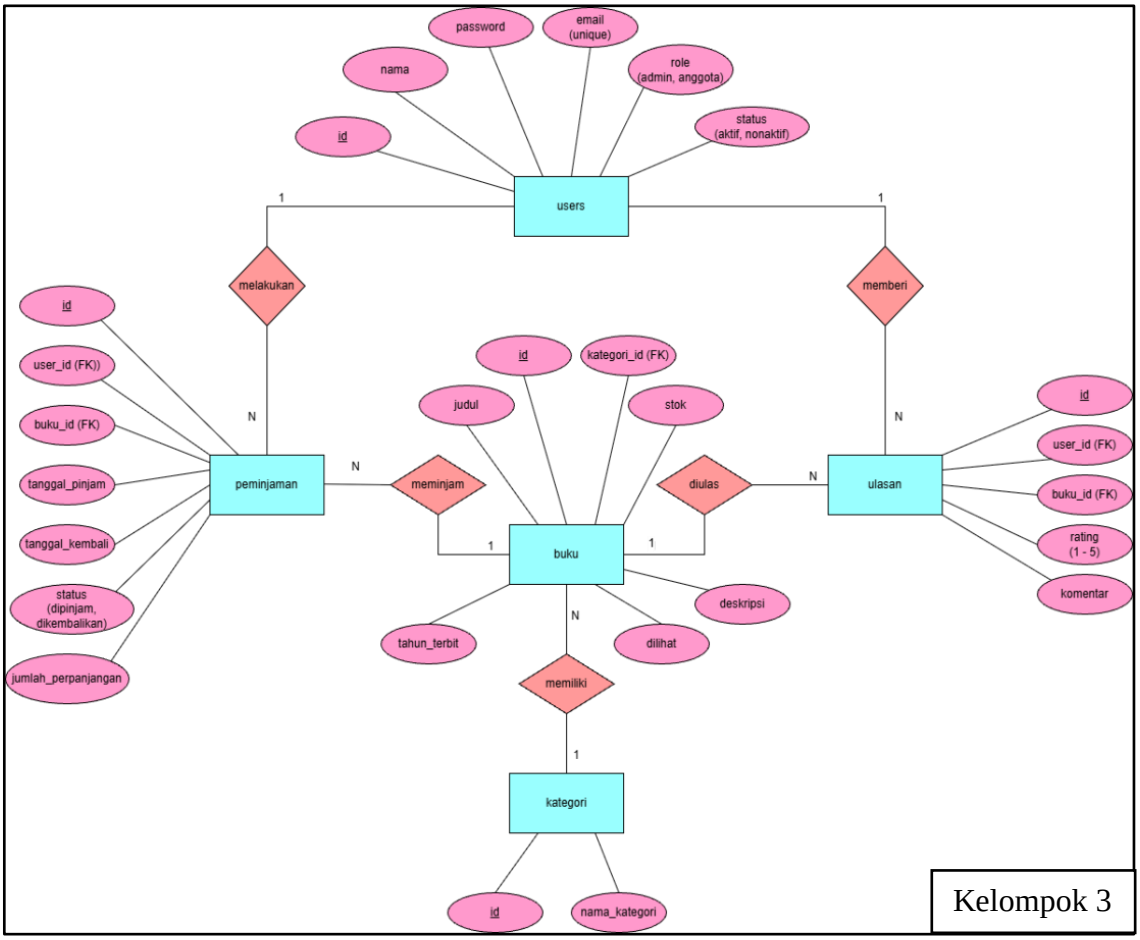
1. Buatlah Sistem Informasi Perpustakaan Digital :

   Backend menggunakan database MYSql

   Frontend bebas boleh CSS, Bootstrap, tailwind dll

   Minimal menggunakan 2 Aktor

   Dalam merancang sebuah sistem informasi, diperlukan pembuatan diagram ERD (*Entity Relationship Diagram*). ERD yang dibuat menggambarkan struktur data pada Sistem Informasi Perpustakaan Digital yang terdiri dari empat entitas utama, yaitu Users, Buku, Peminjaman, dan Kategori, serta satu entitas pendukung yaitu Ulasan. Berikut ini adalah rancangan ERD untuk sistem informasi perpustakaan tersebut:



Gambar 2.1 ERD SI Perpustakaan Digital

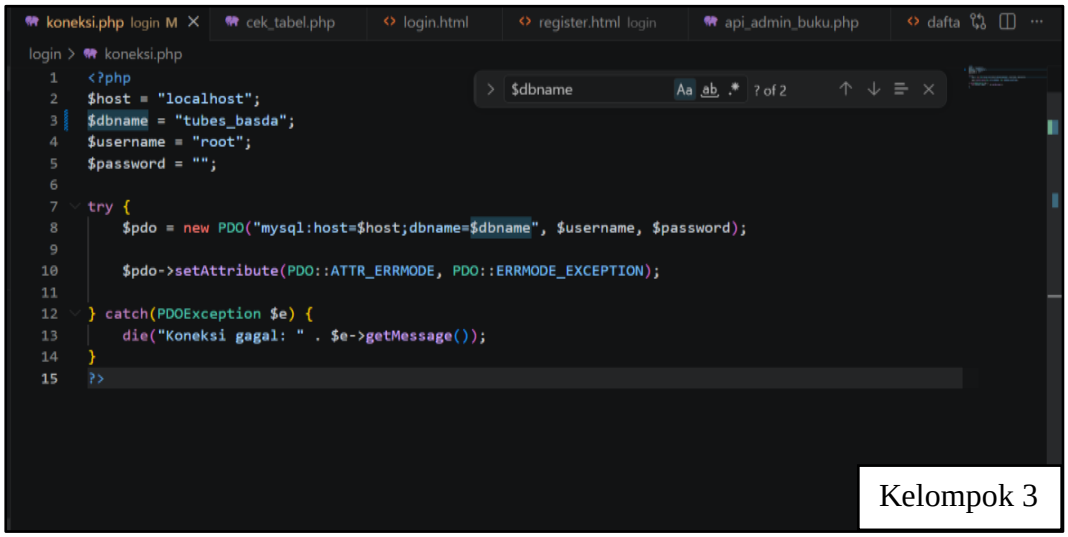
Pada gambar ERD Sistem Informasi Perpustakaan Digital diatas, terdapat entitas utama yang saling berelasi. Pertama, entitas Users merepresentasikan pengguna sistem yang terbagi menjadi dua peran, yaitu admin dan anggota. Entitas ini memiliki beberapa atribut, yaitu id sebagai *primary key*, nama, email (bersifat unik), password, role (admin/anggota), dan status (aktif/nonaktif). Entitas Users memiliki hubungan *one to*

many (1..N) dengan entitas Peminjaman, yang berarti satu pengguna dapat melakukan banyak transaksi peminjaman buku. Selain itu, Users juga memiliki relasi *one to many* (1..N) dengan entitas Ulasan, di mana satu pengguna dapat memberikan banyak ulasan terhadap buku-buku yang telah dibaca.

Entitas Peminjaman berfungsi untuk mencatat aktivitas transaksi peminjaman buku. Atribut-atributnya meliputi id sebagai *primary key*, user_id dan buku_id sebagai *foreign key*, tanggal_pinjam, tanggal_kembali, status (dipinjam/dikembalikan), serta jumlah_perpanjangan. Entitas ini memiliki relasi *many to one* (N..1) dengan entitas Buku, yang menunjukkan bahwa banyak catatan peminjaman dapat merujuk pada satu judul buku yang sama. Entitas Buku merupakan pusat informasi dari koleksi perpustakaan. Atributnya terdiri dari id sebagai *primary key*, judul, penulis, tahun_terbit, stok, deskripsi, dilihat (jumlah *view*), serta kategori_id sebagai *foreign key*. Entitas Buku memiliki relasi *many to one* (N..1) dengan entitas Kategori, karena banyak buku dapat dikelompokkan ke dalam satu kategori yang sama. Di sisi lain, entitas Buku juga berhubungan *one to many* (1..N) dengan entitas Ulasan, yang berarti satu buku dapat menerima banyak ulasan dan rating dari berbagai pengguna.

Entitas Ulasan digunakan untuk menyimpan umpan balik dari pengguna. Atributnya meliputi id, user_id, buku_id, rating (skala 1-5), dan komentar. Terakhir, entitas Kategori berfungsi untuk mengklasifikasikan jenis buku dengan atribut id sebagai *primary key* dan nama_kategori. Secara keseluruhan, hubungan antar entitas pada sistem ini menggambarkan proses sirkulasi buku, mulai dari pengkategorian buku, aktivitas peminjaman oleh pengguna, hingga pemberian ulasan, di mana semua data terintegrasi untuk memudahkan pengelolaan informasi perpustakaan secara digital.

Printscreen source code:



Gambar 2.2 Source Code Koneksi Database (koneksi.php)

Source code:

```
<?php
$host = "localhost";
$dbname = "tubes_basda";
$username = "root";
$password = "";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);

    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

} catch(PDOException $e) {
    die("Koneksi gagal: " . $e->getMessage());
}
?>
```

Penjelasan Source code:

Kode PHP tersebut merupakan modul konfigurasi utama yang berfungsi untuk membangun komunikasi antara aplikasi *web* dan *server* MySQL menggunakan ekstensi PDO (PHP Data Objects). Proses diawali dengan pendeklarasian variabel konfigurasi server, yaitu $host = "localhost"; sebagai alamat *host* lokal, $dbname = "tubes_basda"; sebagai identitas basis data yang dituju, serta $username = "root"; dan $password = ""; sebagai kredensial autentikasi pengguna. Seluruh instruksi koneksi dibungkus di dalam blok try { ... } untuk menangani kesalahan secara dinamis saat waktu jalan (*runtime*). Inti dari penghubungan database dilakukan melalui perintah $pdo = new PDO("mysql:host=$host;dbname=$dbname", $username, $password); yang menciptakan objek PDO baru sebagai jembatan data yang menghubungkan aplikasi dengan database. Agar pengembang dapat melacak kesalahan logika pada kueri SQL dengan mudah, kode $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION); ditambahkan untuk mengaktifkan mode pelaporan pengecualian (*exception*). Apabila sistem gagal membangun koneksi karena parameter yang tidak valid atau server mati, maka blok catch(PDOException $e) akan menangkap pesan kesalahan tersebut dan mengeksekusi fungsi die("Koneksi gagal: " . $e->getMessage()); untuk menghentikan seluruh aktivitas aplikasi dan juga menampilkan pesan kesalahannya demi keamanan sistem.

Printscreen source code:

Gambar 2.3 Source Code Pengujian Koneksi Database (cek_tabel.php)

Source code:

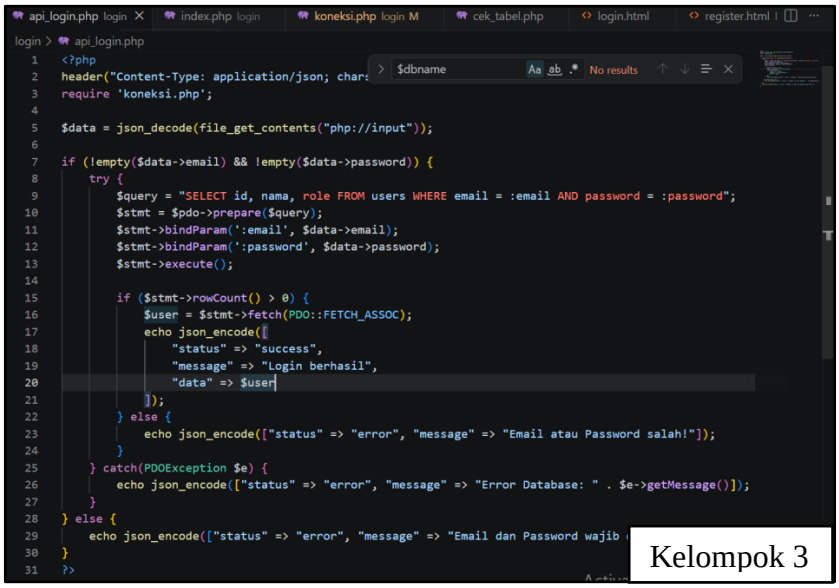
```
<?php
require 'koneksi.php';

try {
    $query = $pdo->query("SELECT * FROM users");
    $data = $query->fetchAll(PDO::FETCH_ASSOC);

    echo "<h1>Debug Database (Folder Login)</h1>";
    if (empty($data)) {
        echo "<p style='color:red'>Data KOSONG! PHP sudah konek, tapi di tabel users belum ada isinya.</p>";
    } else {
        echo "<pre>";
        print_r($data);
        echo "</pre>";
    }
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
}
?>
```

Penjelasan Source code:

Modul cek_tabel.php berfungsi sebagai instrumen pengujian (*debugging*) internal untuk memverifikasi keberhasilan integrasi antara aplikasi web dan basis data MySQL.

Setelah memanggil file konfigurasi utama melalui instruksi require 'koneksi.php', skrip ini mencoba mengeksekusi kueri dasar SELECT * FROM users menggunakan objek PDO. Keunggulan dari skrip pengujian ini adalah penerapan blok try-catch serta percabangan if-else; di mana jika tabel target masih kosong atau terjadi galat (*error*) pada server, sistem akan memberikan pesan umpan balik visual (*feedback*) berupa teks peringatan, alih-alih membiarkan aplikasi mengalami *crash* secara diam-diam. Langkah validasi awal ini sangat krusial untuk memastikan stabilitas jalur komunikasi data sebelum sistem mengimplementasikan operasi CRUD tingkat lanjut.

Printscreen source code:



Gambar 2.4 Source Code Autentikasi Login (api_login.php)

Source code:

```
<?php
header("Content-Type: application/json; charset=UTF-8");
require 'koneksi.php';

$data = json_decode(file_get_contents("php://input"));

if (!empty($data->email) && !empty($data->password)) {
    try {
        $query = "SELECT id, nama, role FROM users WHERE email = :email AND
password = :password";
        $stmt = $pdo->prepare($query);
        $stmt->bindParam(':email', $data->email);
        $stmt->bindParam(':password', $data->password);
```

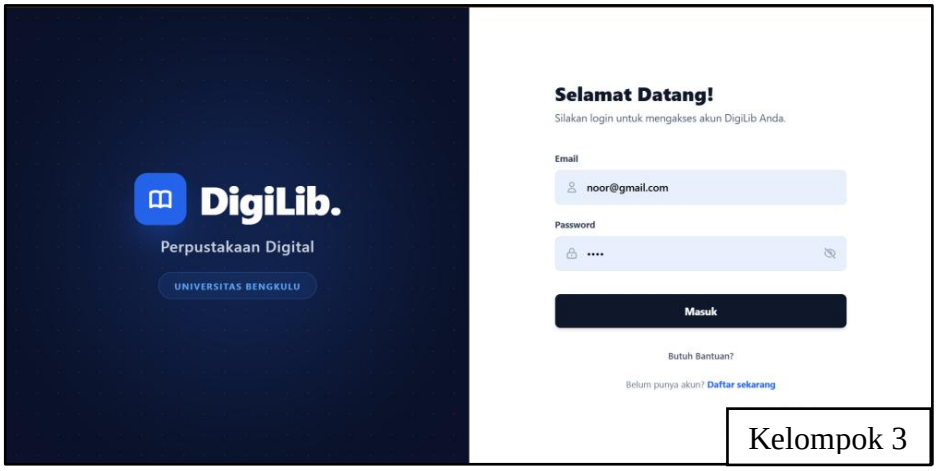
```
        $stmt->execute();

        if ($stmt->rowCount() > 0) {
            $user = $stmt->fetch(PDO::FETCH_ASSOC);
            echo json_encode([
                "status" => "success",
                "message" => "Login berhasil",
                "data" => $user
            ]);
        } else {
            echo  json_encode(["status"  =>  "error",  "message"  =>  "Email  atau  Password
salah!"]);
        }
    } catch(PDOException $e) {
        echo  json_encode(["status"  =>  "error",  "message"  =>  "Error  Database:  "  .  $e-
>getMessage()]);
    }
} else {
    echo  json_encode(["status"  =>  "error",  "message"  =>  "Email  dan  Password  wajib
diisi!"]);
}
?>
```

Pembahasan Source Code :

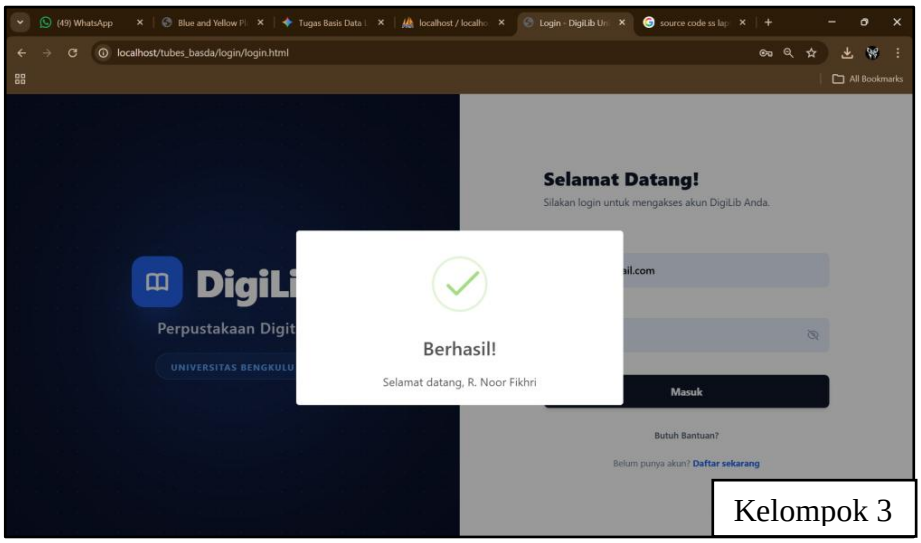
File api_login.php berfungsi sebagai gerbang autentikasi yang memproses kredensial pengguna secara dinamis dengan format data JSON. Program diawali dengan memanggil modul koneksi melalui perintah require 'koneksi.php'; agar dapat berinteraksi dengan database. Data kiriman dari *frontend* ditangkap menggunakan instruksi json_decode(file_get_contents("php://input")), yang kemudian divalidasi untuk memastikan email dan password tidak kosong. Inti dari proses pencarian data terletak pada kueri DQL (*Data Query Language*) yaitu $query = "SELECT id, nama, role FROM users WHERE email = :email AND password = :password";. Untuk menjaga keamanan dari serangan *SQL Injection*, sistem menerapkan *prepared statement* melalui fungsi $pdo->prepare($query) dan pengikatan parameter dengan $stmt->bindParam(). Setelah kueri dijalankan menggunakan $stmt->execute(), sistem akan mengecek keberadaan data melalui $stmt->rowCount() > 0. Jika data ditemukan, informasi pengguna akan diambil dengan

$stmt->fetch(PDO::FETCH_ASSOC) dan dikirimkan kembali ke aplikasi dalam format JSON menggunakan echo json_encode(...) dengan status sukses. Sebaliknya, jika data tidak cocok atau terjadi kendala pada server, blok catch(PDOException $e) akan menangkap pengecualian tersebut dan mengirimkan pesan galat yang sesuai.

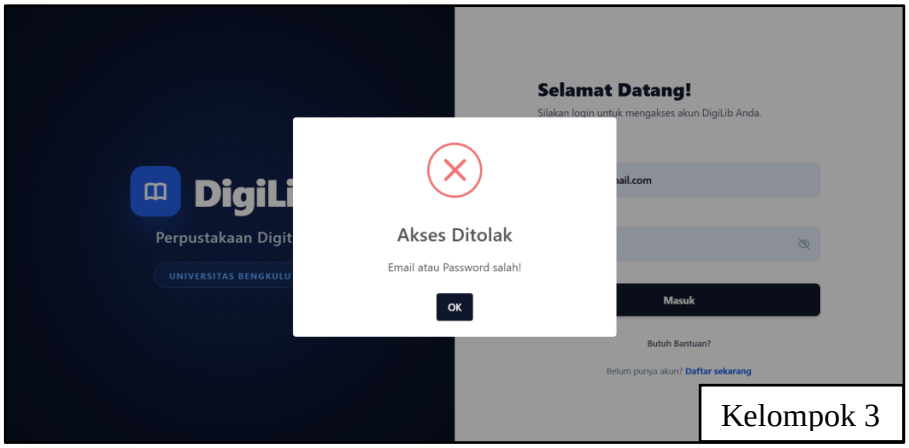
Tampilan SI:



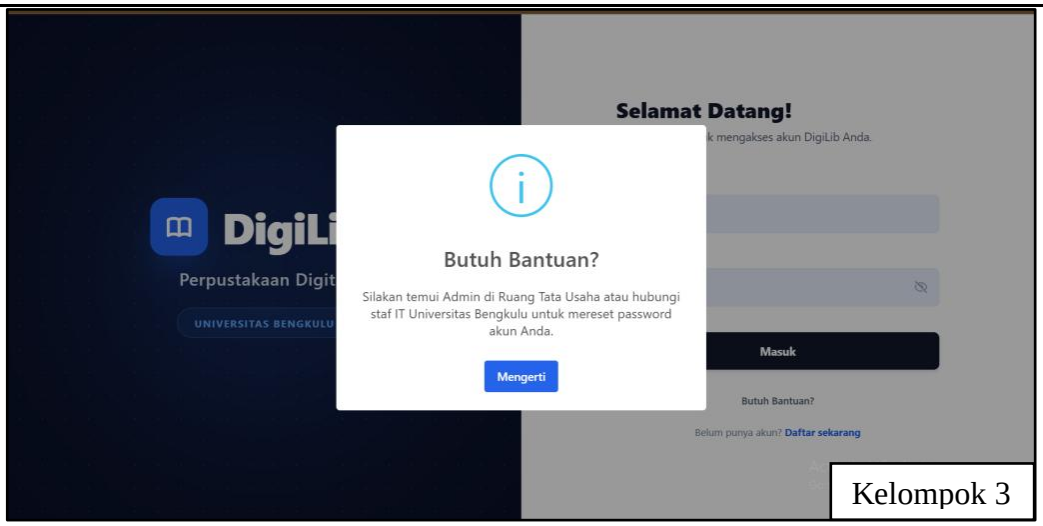
Gambar 2.5 Tampilan Form Input Login



Gambar 2.6 Tampilan Autentikasi Berhasil



Gambar 2.7 Output Autentikasi Gagal (Akses Ditolak)



Gambar 2.8 Tampilan Pop-up Bantuan Login

Penjelasan Tampilan SI:

Rangkaian gambar di atas merepresentasikan siklus pengujian fungsionalitas pada fitur autentikasi sistem DigiLib, mulai dari tahap pemasukan data hingga respon balik dari server. Proses dimulai pada halaman antarmuka login di mana pengguna memasukkan kredensial berupa email dan kata sandi yang akan dikirimkan ke backend dalam format JSON. Apabila data yang diinputkan sesuai dengan record yang tersimpan pada tabel users, sistem akan memberikan respon sukses melalui kemunculan jendela pop-up "Berhasil!" yang menyapa nama pengguna secara spesifik, menandakan bahwa kueri SELECT telah berhasil menemukan kecocokan data. Sebaliknya, jika pengguna memasukkan kombinasi email atau kata sandi yang salah, mekanisme validasi pada kode PHP akan menangkap kegagalan pencarian data dan menampilkan notifikasi "Akses Ditolak". Pengujian ini membuktikan bahwa logika percabangan pada sistem autentikasi telah berjalan secara akurat dalam membedakan akses sah dan tidak sah, sekaligus menjaga keamanan integritas basis data perpustakaan digital. Pop-up Bantuan yang berfungsi sebagai fitur pendukung pada halaman autentikasi. Jendela informasi ini akan muncul secara dinamis saat pengguna mengeklik tautan "Butuh Bantuan?" jika mengalami kendala akses seperti lupa kata sandi. Melalui fitur ini, sistem memberikan instruksi prosedural bagi pengguna untuk menghubungi administrator atau staf IT Universitas Bengkulu guna melakukan reset akun. Penambahan elemen interaktif ini membuktikan bahwa aplikasi DigiLib dirancang dengan memperhatikan aspek User Experience (UX), guna memastikan setiap anggota mendapatkan solusi teknis secara instan tanpa harus meninggalkan halaman utama sistem. Integrasi fitur ini secara langsung meningkatkan kualitas layanan perpustakaan digital dan memberikan kemudahan bagi semua pengguna serta meminimalisir kendala teknis saat login.

Printscreen source code:

Gambar 2.9 Source Code Pendaftaran Akun Baru (api_register.php)

Source code:

```
<?php
header("Content-Type: application/json; charset=UTF-8");
require 'koneksi.php';

$data = json_decode(file_get_contents("php://input"));

if (!empty($data->nama) && !empty($data->email) && !empty($data->password)) {
    try {
        $check = $pdo->prepare("SELECT email FROM users WHERE email = :email");
        $check->execute(['email' => $data->email]);

        if($check->rowCount() > 0) {
            echo json_encode(["status" => "error", "message" => "Email sudah digunakan! Silakan gunakan email lain."]);
            exit;
        }


        $role = (!empty($data->role) && $data->role == 'admin') ? 'admin' : 'anggota';

        $query = "INSERT INTO users (nama, email, password, role, status) VALUES (:nama, :email, :password, :role, 'aktif')";
```

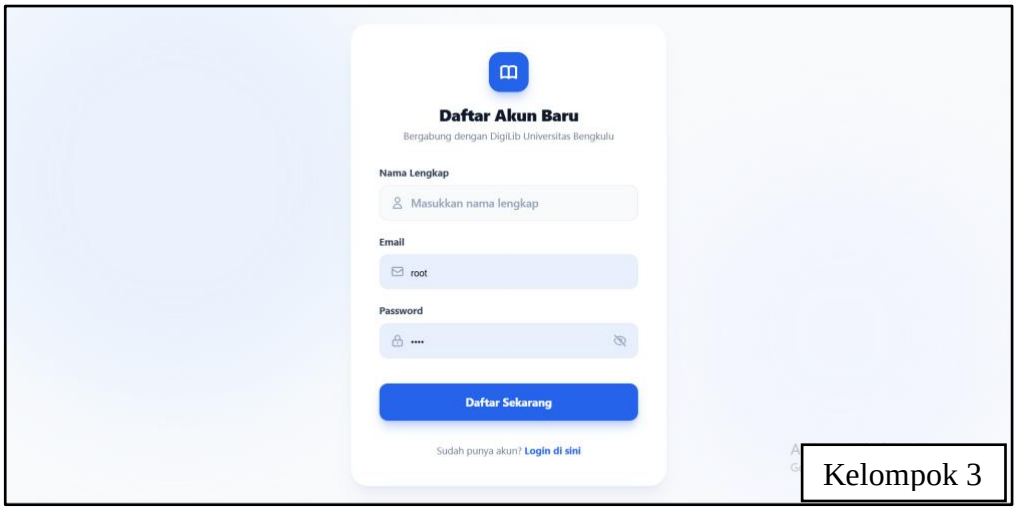
```
        $stmt = $pdo->prepare($query);
        $stmt->bindParam(':nama', $data->nama);
        $stmt->bindParam(':email', $data->email);
        $stmt->bindParam(':password', $data->password);
        $stmt->bindParam(':role', $role);

        if ($stmt->execute()) {
            echo json_encode(["status" => "success", "message" => "Pendaftaran berhasil! Silakan Login."]);
        } else {
            echo json_encode(["status" => "error", "message" => "Gagal menyimpan data ke database."]);
        }
    } catch(PDOException $e) {
        echo json_encode(["status" => "error", "message" => "Error Database: " . $e->getMessage()]);
    }
} else {
    echo json_encode(["status" => "error", "message" => "Semua kolom wajib diisi!"]);
}
?>
```

Pembahasan Source Code :

Modul api_register.php berfungsi untuk menangani proses pendaftaran anggota baru ke dalam sistem perpustakaan digital secara dinamis. Skrip ini menerima data JSON dari antarmuka formulir registrasi yang mencakup nama, email, dan kata sandi. Sebelum menyimpan data, sistem menerapkan algoritma validasi dengan mengeksekusi kueri DQL (*Data Query Language*) yaitu SELECT email FROM users WHERE email = :email. Langkah preventif ini sangat esensial untuk memastikan tidak terjadinya duplikasi identitas; jika email sudah terdaftar, sistem akan langsung menghentikan proses dan mengembalikan pesan *error* . Apabila email valid dan belum digunakan, sistem melanjutkan proses dengan mengeksekusi kueri DML INSERT INTO users (nama, email, password, role, status) VALUES (:nama, :email, :password, :role, 'aktif'). Kode ini menggunakan struktur *prepared statement* (PDO) untuk mencegah potensi manipulasi data melalui *SQL Injection*. Secara otomatis, sistem menggunakan *ternary operator* PHP untuk mengatur hak akses (*role*) pengguna baru sebagai 'anggota' dengan status langsung 'aktif',

sehingga akun tersebut langsung siap digunakan untuk masuk (login) ke dalam aplikasi perpustakaan.

Tampilan SI:

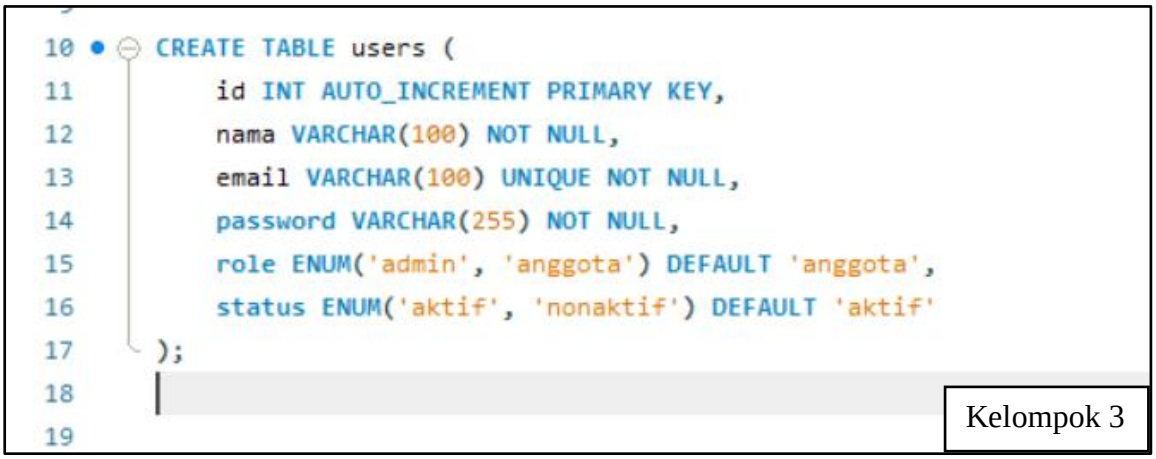


Gambar 2.10 Antarmuka Formulir Daftar Akun Baru

Penjelasan tampilan SI:

Gambar 2.11 menampilkan antarmuka halaman registrasi yang didesain secara minimalis untuk memudahkan calon anggota dalam melakukan pendaftaran. Administrator atau pengguna baru cukup memasukkan Nama Lengkap, Email, dan Kata Sandi pada kolom yang tersedia. Tampilan ini juga dilengkapi dengan tautan navigasi balik "Login di sini" untuk memberikan pengalaman pengguna (User Experience) yang lebih baik jika ternyata pengguna sudah memiliki akun sebelumnya guna mempercepat proses registrasi akun baru.

Printscreen source code:



Gambar 2.11 Source Code Membuat Tabel Users

Source code:

```
CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nama VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role ENUM('admin', 'anggota') DEFAULT 'anggota',
    status ENUM('aktif', 'nonaktif') DEFAULT 'aktif'
);
```

Penjelasan Source code:

Source code di atas digunakan untuk membuat tabel users yang berfungsi menyimpan data akun pengguna, baik untuk level admin maupun anggota, pada sistem perpustakaan digital agar proses manajemen peminjaman buku dapat berjalan dengan lebih efisien. Dalam tabel ini terdapat kolom id dengan fitur AUTO_INCREMENT sebagai *primary key* untuk memberikan identitas unik pada setiap pengguna secara otomatis dan berurutan dari angka terkecil. Kolom nama digunakan untuk menyimpan nama lengkap pengguna, email digunakan sebagai identitas login yang bersifat UNIQUE (tidak boleh sama antar pengguna), dan password untuk menyimpan kata sandi akun yang sangat penting bagi keamanan aplikasi. Selain itu, terdapat kolom role dengan tipe data ENUM yang membatasi peran pengguna hanya sebagai 'admin' atau 'anggota', serta kolom status untuk menentukan apakah akun tersebut sedang dalam keadaan 'aktif' atau 'nonaktif'. Kedua kolom terakhir memiliki nilai DEFAULT, sehingga jika tidak diisi saat pendaftaran, sistem akan otomatis mengaturnya sebagai 'anggota' dengan status 'aktif' hal ini sangat memudahkan proses pendaftaran tanpa perlu input manual.

Printscreen source code:

```
67 ●  INSERT INTO users (nama, email, password, role, status) VALUES
68    ('R. Noor Fikhri', 'noor@gmail.com', '1234', 'admin', 'aktif'),
69    ('Selda Abriana', 'selda@gmail.com', '1234', 'anggota', 'aktif'),
70    ('Tri Haiji Januarli ', 'haiji@gmail.com', '1234', 'anggota', 'aktif'),
71    ('Ikhsan Rihan Farli', 'ikhsan@gmail.com', '1234', 'anggota', 'aktif');
72
```

Kelompok 3

Gambar 2.12 Tambah Data Users

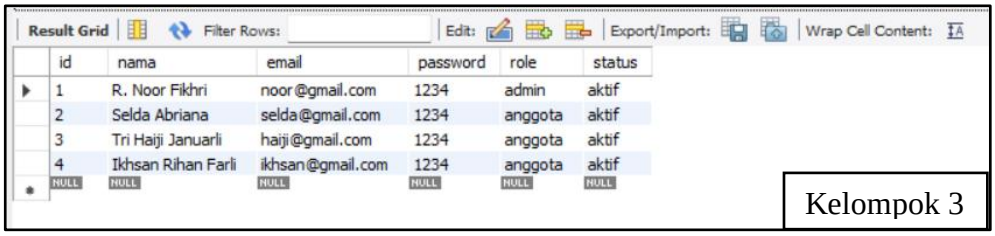
Source code:

```
INSERT INTO users (nama, email, password, role, status) VALUES
('R. Noor Fikhri', 'noor@gmail.com', '1234', 'admin', 'aktif'),
('Selda Abriana', 'selda@gmail.com', '1234', 'anggota', 'aktif'),
('Tri Haiji Januarli ', 'haiji@gmail.com', '1234', 'anggota', 'aktif'),
('Ikhsan Rihan Farli', 'ikhsan@gmail.com', '1234', 'anggota', 'aktif');
```

Pembahasan Source Code :

Source code di atas menggunakan perintah INSERT INTO yang berfungsi untuk menambahkan data awal (*seeding data*) ke dalam tabel users. Data yang dimasukkan mencakup kolom nama, email, password, role, dan status untuk empat pengguna berbeda. Pengguna pertama, R. Noor Fikhri, didaftarkan dengan peran sebagai 'admin' yang memiliki hak akses penuh untuk mengelola sistem. Selanjutnya, terdapat tiga data pengguna lain, yaitu Selda Abriana, Tri Haiji Januarli, dan Ikhsan Rihan Farli, yang didaftarkan dengan peran sebagai 'anggota'. Semua akun tersebut diatur dengan status 'aktif' dan menggunakan kata sandi '1234'. Penambahan data awal ini bertujuan agar sistem memiliki akun siap pakai yang dapat digunakan untuk proses pengujian fungsi *login* serta fitur-fitur lainnya pada aplikasi perpustakaan digital.

Printscreen Output:



Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	nama	email	password	role	status
1	R. Noor Fikhri	noor@gmail.com	1234	admin	aktif
2	Selda Abriana	selda@gmail.com	1234	anggota	aktif
3	Tri Haiji Januarli	haiji@gmail.com	1234	anggota	aktif
4	Ikhsan Rihan Farli	ikhsan@gmail.com	1234	anggota	aktif
NULL	NULL	NULL	NULL	NULL	NULL

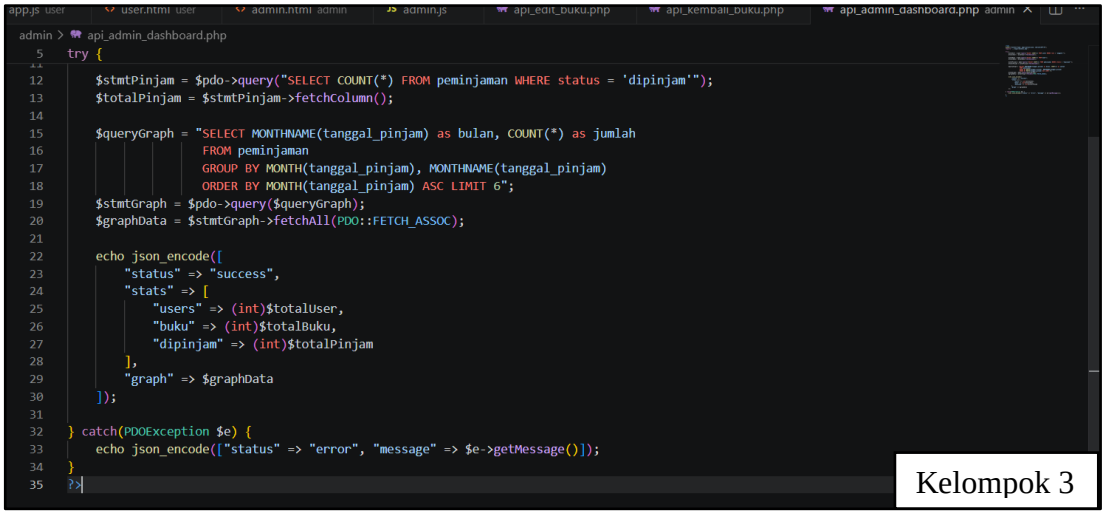
Kelompok 3

Gambar 2.13 Tabel Users

Penjelasan Output:

Gambar di atas menampilkan isi data dari tabel users pada database perpustakaan_digital. Tabel ini berfungsi untuk menyimpan seluruh informasi akun pengguna yang terdaftar dalam Sistem Informasi Perpustakaan Digital. Dalam tampilan tersebut, terlihat kolom id sebagai identitas unik setiap pengguna, nama yang menyimpan identitas lengkap pengguna, email sebagai alamat surat elektronik untuk keperluan akun, serta password untuk keamanan akses. Data tersebut juga memperlihatkan pembagian hak akses melalui kolom role, di mana terdapat akun dengan peran sebagai 'admin' dan 'anggota', serta kolom status yang menunjukkan bahwa seluruh akun dalam daftar tersebut saat ini berada dalam keadaan 'aktif'. Tabel ini memiliki peran krusial dalam sistem autentikasi, yang memastikan setiap pengguna mendapatkan akses dan fitur yang sesuai dengan peran mereka masing-masing saat menggunakan aplikasi. Keberadaan data yang valid pada tabel ini menjadi fondasi utama bagi terjaganya integritas serta keamanan seluruh riwayat transaksi di dalam sistem DigiLib. Selain itu, struktur data yang terorganisir ini memudahkan administrator dalam melakukan pemantauan serta manajemen akun pengguna secara efisien melalui panel kontrol aplikasi demi kelancaran akses seluruh anggota terdaftar.

Printscreen source code:




```
try {
    $stmtPinjam = $pdo->query("SELECT COUNT(*) FROM peminjaman WHERE status = 'dipinjam'");
    $totalPinjam = $stmtPinjam->fetchColumn();

    $queryGraph = "SELECT MONTHNAME(tanggal_pinjam) as bulan, COUNT(*) as jumlah
                   FROM peminjaman
                   GROUP BY MONTH(tanggal_pinjam), MONTHNAME(tanggal_pinjam)
                   ORDER BY MONTH(tanggal_pinjam) ASC LIMIT 6";
    $stmtGraph = $pdo->query($queryGraph);
    $graphData = $stmtGraph->fetchAll(PDO::FETCH_ASSOC);

    echo json_encode([
        "status" => "success",
        "stats" => [
            "users" => (int)$totalUser,
            "buku" => (int)$totalBuku,
            "dipinjam" => (int)$totalPinjam
        ],
        "graph" => $graphData
    ]);

} catch(PDOException $e) {
    echo json_encode(["status" => "error", "message" => $e->getMessage()]);
}
?>
```

Gambar 2.14 Source Code Logika Dashboard Admin (api_admin_dashboard.php)

Source code:

```
<?php
header("Content-Type: application/json; charset=UTF-8");
require '../login/koneksi.php';

try {
    $stmtUser = $pdo->query("SELECT COUNT(*) FROM users WHERE role = 'anggota'");
    $totalUser = $stmtUser->fetchColumn();

    $stmtBuku = $pdo->query("SELECT COUNT(*) FROM buku");
    $totalBuku = $stmtBuku->fetchColumn();

    $stmtPinjam = $pdo->query("SELECT COUNT(*) FROM peminjaman WHERE status = 'dipinjam'");
    $totalPinjam = $stmtPinjam->fetchColumn();

    $queryGraph = "SELECT MONTHNAME(tanggal_pinjam) as bulan, COUNT(*) as jumlah
                   FROM peminjaman
                   GROUP BY MONTH(tanggal_pinjam), MONTHNAME(tanggal_pinjam)
                   ORDER BY MONTH(tanggal_pinjam) ASC LIMIT 6";
    $stmtGraph = $pdo->query($queryGraph);
```

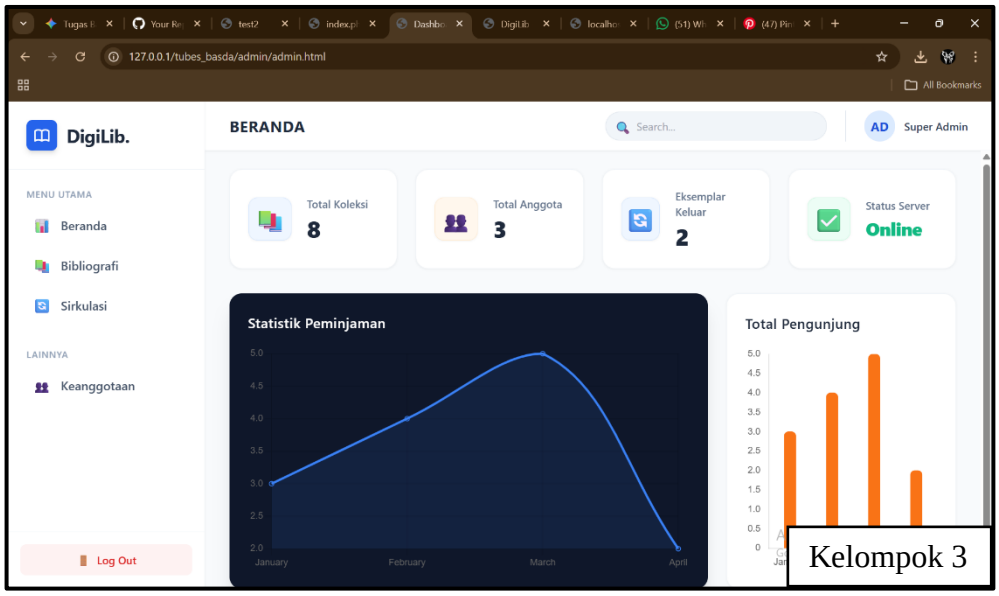
```
    $graphData = $stmtGraph->fetchAll(PDO::FETCH_ASSOC);

    echo json_encode([
        "status" => "success",
        "stats" => [
            "users" => (int)$totalUser,
            "buku" => (int)$totalBuku,
            "dipinjam" => (int)$totalPinjam
        ],
        "graph" => $graphData
    ]);

} catch(PDOException $e) {
    echo json_encode(["status" => "error", "message" => $e->getMessage()]);
}
?>
```

Pembahasan Source Code :

Modul api_admin_dashboard.php bertanggung jawab untuk menyuplai data analitik yang ditampilkan pada halaman utama administrator. Skrip ini menonjolkan penggunaan fungsi agregasi SQL tingkat lanjut untuk menghasilkan ringkasan data secara otomatis. Sistem menjalankan beberapa kueri secara simultan, di antaranya adalah SELECT COUNT(*) pada tabel users, buku, dan peminjaman untuk mendapatkan total data secara *real-time*.

Selain itu, modul ini mengimplementasikan kueri yang lebih kompleks untuk kebutuhan visualisasi grafik, yaitu SELECT MONTHNAME(tanggal_pinjam) as bulan, COUNT(*) as jumlah FROM peminjaman GROUP BY MONTH(tanggal_pinjam). Kueri tersebut mendemonstrasikan kemampuan sistem dalam melakukan pengelompokan data (*Grouping*) berdasarkan fungsi kronologis dan mengurutkannya secara kronologis menggunakan klausa ORDER BY. Data yang dihasilkan kemudian dipaketkan ke dalam objek JSON terstruktur yang mencakup statistik numerik dan *array* data grafik. Pendekatan ini menjamin bahwa informasi yang disajikan kepada administrator selalu akurat dan mencerminkan kondisi terkini dari basis data perpustakaan digital. Keberadaan data analitik yang terorganisir ini memberikan kemudahan bagi administrator dalam memantau dinamika aktivitas perpustakaan serta mengambil keputusan strategis berdasarkan tren peminjaman yang valid secara efisien dan tepat sasaran.

Tampilan SI:



Gambar 2.15 Tampilan Halaman Beranda (Sisi Admin)

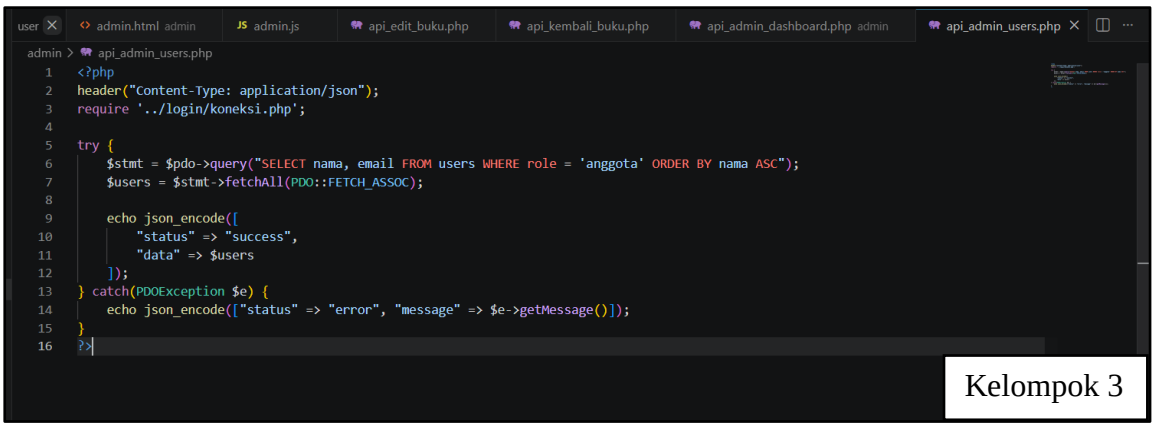
Penjelasan Tampilan SI:

Gambar di atas menampilkan halaman Beranda atau *Dashboard* utama bagi pengguna dengan hak akses Super Admin pada ekosistem website DigiLib. Halaman ini dirancang secara khusus sebagai pusat kendali operasional dan pemantauan ringkas mengenai kondisi perpustakaan secara *real-time*. Pada bagian *header*, terdapat deretan kartu informasi (*infographic cards*) yang menyajikan metrik kunci secara instan, seperti Total Koleksi yang mencakup seluruh judul buku, Total Anggota yang terdaftar, jumlah Eksemplar Keluar untuk melacak buku yang sedang dalam masa peminjaman, serta indikator Status Server untuk memastikan sistem berjalan tanpa kendala teknis.

Selain ringkasan data statistik, halaman ini dilengkapi dengan visualisasi data interaktif berupa grafik garis untuk Statistik Peminjaman dan grafik batang untuk Total Pengunjung dalam periode Januari hingga April. Kehadiran grafik ini sangat membantu admin dalam menganalisis tren aktivitas literasi pengguna serta memudahkan proses pengambilan keputusan manajerial berbasis data (*data-driven decision making*).

Di sisi kiri antarmuka, tersedia menu navigasi terintegrasi yang mencakup modul Bibliografi, Sirkulasi, hingga Keanggotaan, yang disusun secara modular untuk mempermudah alur kerja admin. Seluruh fungsionalitas ini ditutup dengan tombol *Log Out* yang memastikan sesi akses admin dapat diakhiri dengan prosedur keamanan yang tepat demi menjaga kerahasiaan data perpustakaan. Dengan integrasi seluruh fitur tersebut, administrator dapat mengelola ekosistem DigiLib secara lebih terstruktur, cepat, dan transparan melalui satu gerbang kendali utama demi efisiensi pengelolaan perpustakaan digital..

Printscreen source code:



Gambar 2.16 Source Code Logika Pengambilan Data Anggota (api_admin_users.php)

Source code:

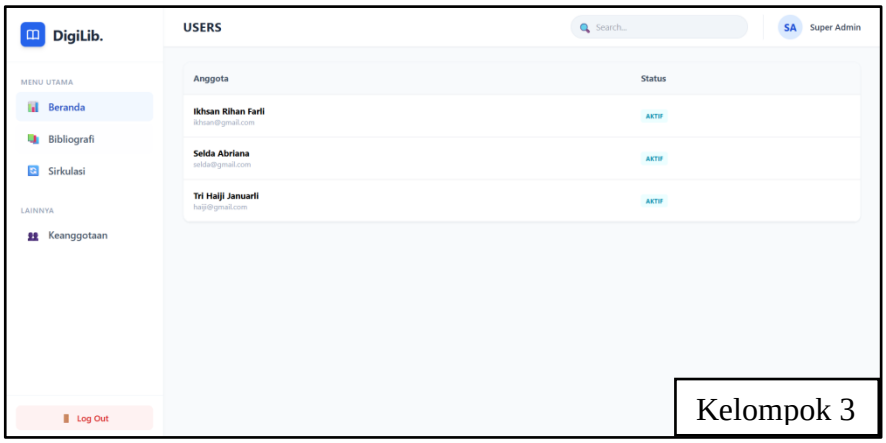
```
<?php
header("Content-Type: application/json");
require '../login/koneksi.php';

try {
   $stmt = $pdo->query("SELECT nama, email FROM users WHERE role = 'anggota' ORDER BY nama ASC");
   $users = $stmt->fetchAll(PDO::FETCH_ASSOC);

   echo json_encode([
      "status" => "success",
      "data" => $users
   ]);
} catch(PDOException $e) {
   echo json_encode(["status" => "error", "message" => $e->getMessage()]);
}
?>
```

Penjelasan Source code:

Rangkaian gambar di atas merepresentasikan fungsionalitas manajemen pengguna, khususnya kapabilitas sistem dalam menyajikan daftar keanggotaan secara spesifik kepada administrator. Pada lapisan *backend*, modul api_admin_users.php bertugas mengekstraksi data identitas dengan mengeksekusi kueri DQL terfilter yaitu SELECT nama, email FROM users WHERE role = 'anggota' ORDER BY nama ASC. Penggunaan klausa kondisional WHERE role = 'anggota' merupakan bentuk filterisasi krusial untuk memastikan bahwa

sistem hanya menampilkan akun pengguna biasa dan menyembunyikan kredensial sesama administrator dari daftar ini. Selanjutnya, data tersebut diurutkan secara alfabetis menggunakan ORDER BY nama ASC untuk mempermudah proses pencarian manual. Hasil kueri kemudian diproses melalui metode $stmt->fetchAll(PDO::FETCH_ASSOC) dan dikembalikan kepada *browser* klien melalui perintah echo json_encode(["status" => "success", "data" => $users]). Pada sisi *frontend*, paket data JSON tersebut diurai dan diproyeksikan ke dalam antarmuka tabel "Data Anggota Perpustakaan" secara *real-time*. Tabel ini menyajikan informasi Nama Lengkap dan Email secara terstruktur, memberikan kendali visual yang komprehensif bagi administrator untuk melakukan audit terhadap basis pengguna aktif yang terdaftar di dalam sistem DigiLib.

Tampilan SI:



Gambar 2.17 Antarmuka Keanggotaan (Sisi Admin)

Penjelasan Tampilan SI:

Gambar di atas merepresentasikan antarmuka pengguna (*User Interface*) yang dirender secara eksklusif untuk entitas dengan tingkat otorisasi 'anggota'. Pada halaman Dasbor ini, sistem menerapkan filterisasi privasi data yang ketat dengan memanfaatkan variabel sesi melalui klausa kondisional WHERE user_id = ? yang secara dinamis diikatkan pada identitas $_SESSION['user_id']. Untuk menyuplai data pada panel statistik, lapisan *backend* mengeksekusi fungsi agregasi seperti SELECT COUNT(*) FROM peminjaman WHERE user_id = ? AND status = 'dipinjam' guna menghasilkan rekapitulasi aktivitas sirkulasi personal secara *real-time*. Sementara itu, untuk menyajikan tabel "Riwayat Peminjaman Saya", sistem memproses kueri relasional tingkat lanjut yaitu SELECT b.judul, p.tanggal_pinjam, p.tanggal_kembali, p.status FROM peminjaman p JOIN buku b ON p.buku_id = b.id WHERE p.user_id = ? ORDER BY p.tanggal_pinjam DESC. Melalui pendekatan arsitektur ini, anggota dapat memantau secara mandiri informasi profil, statistik interaksi, hingga rekam jejak peminjaman mereka secara transparan, namun dibatasi hanya pada kapasitas baca (*Read-Only*), yang mana

implementasi ini sangat mencerminkan penerapan prinsip keamanan integritas data pada sistem perpustakaan digital.

Printscreen source code:

```
20 • CREATE TABLE buku (
21       id INT AUTO_INCREMENT PRIMARY KEY,
22       kategori_id INT,
23       judul VARCHAR(200) NOT NULL,
24       penulis VARCHAR(100) NOT NULL,
25       tahun_terbit YEAR,
26       stok INT DEFAULT 0,
27       cover_url VARCHAR(255),
28       deskripsi TEXT,
29       dilihat INT DEFAULT 0,
30       FOREIGN KEY (kategori_id) REFERENCES kategori(id) ON
31   );
```

Kelompok 3

Gambar 2.18 Source Code Membuat Tabel Buku

Source code:

```
CREATE TABLE buku (
   id INT AUTO_INCREMENT PRIMARY KEY,
   kategori_id INT,
   judul VARCHAR(200) NOT NULL,
   penulis VARCHAR(100) NOT NULL,
   tahun_terbit YEAR,
   stok INT DEFAULT 0,
   cover_url VARCHAR(255),
   deskripsi TEXT,
   dilihat INT DEFAULT 0,
   FOREIGN KEY (kategori_id) REFERENCES kategori(id) ON DELETE SET NULL
);
```

Pembahasan Source Code:

Source code di atas digunakan untuk membuat tabel buku yang berfungsi sebagai database utama untuk menyimpan seluruh informasi koleksi pustaka dalam sistem. Dalam tabel ini, terdapat kolom id sebagai *primary key* dengan fitur AUTO_INCREMENT untuk mengidentifikasi setiap judul buku secara unik. Kolom kategori_id digunakan sebagai *foreign key* yang menghubungkan tabel buku dengan tabel kategori, di mana perintah ON DELETE SET NULL memastikan bahwa jika sebuah kategori dihapus, data buku yang terkait tidak ikut terhapus melainkan hanya dikosongkan nilai kategorinya. Selain itu, tabel ini menyimpan detail bibliografi seperti judul, penulis, dan tahun_terbit. Terdapat juga kolom stok untuk mengelola ketersediaan jumlah fisik buku, cover_url untuk menyimpan alamat gambar sampul, serta deskripsi untuk ringkasan konten buku. Kolom dilihat berfungsi untuk mencatat statistik popularitas buku berdasarkan jumlah

kunjungan pengguna. Struktur tabel ini sangat krusial untuk mendukung fitur pencarian, filter kategori, dan manajemen inventaris pada aplikasi perpustakaan digital..

Printscreen source code:

```
INSERT INTO buku (judul, penulis, kategori_id, tahun_terbit, stok, deskripsi, dilihat) VALUES
-- 1 = Fiksi, 2 = Non Fiksi
('Basis Data Lanjut', 'C.J. Date', 2, 2023, 5, 'Teori mendalam tentang SQL dan optimasi query.', 45),
('Filosofi Teras', 'Henry Manampiring', 2, 2019, 12, 'Penerapan Stoikisme dalam kehidupan modern.', 88),
('Laskar Pelangi', 'Andrea Hirata', 1, 2005, 8, 'Kisah perjuangan 10 anak di Belitong.', 30),
('Python for AI', 'Guido Van Rossum', 2, 2024, 15, 'Dasar pemrograman Python untuk AI.', 120),

('Danur: Gerbang Dialog', 'Risa Saraswati', 3, 2011, 7, 'Kisah persahabatan anak manusia dengan entitas tak kasat mata.', 55),

-- 4 = Romance
('Dilan 1990', 'Pidi Baiq', 4, 2014, 20, 'Kisah romansa masa putih abu-abu di Bandung.', 210),

-- 5 = Fantasi
('Bumi', 'Tere Liye', 5, 2014, 10, 'Petualangan fantasi remaja di dunia paralel Klan Bulan.', 95),

-- 6 = Komik
('The Greatest Estate Developer', 'BK_Moon', 6, 2021, 25, 'Manhwa genre kingdom building tentang mahasiswa teknik sip
```

Kelompok 3

Gambar 2.19 Tambah Data Buku

Source code:

INSERT INTO buku (judul, penulis, kategori_id, tahun_terbit, stok, deskripsi, dilihat) VALUES

-- 1 = Fiksi, 2 = Non Fiksi

('Basis Data Lanjut', 'C.J. Date', 2, 2023, 5, 'Teori mendalam tentang SQL dan optimasi query.', 45),

('Filosofi Teras', 'Henry Manampiring', 2, 2019, 12, 'Penerapan Stoikisme dalam kehidupan modern.', 88),

('Laskar Pelangi', 'Andrea Hirata', 1, 2005, 8, 'Kisah perjuangan 10 anak di Belitong.', 30),

('Python for AI', 'Guido Van Rossum', 2, 2024, 15, 'Dasar pemrograman Python untuk AI.', 120),

('Danur: Gerbang Dialog', 'Risa Saraswati', 3, 2011, 7, 'Kisah persahabatan anak manusia dengan entitas tak kasat mata.', 55),

-- 4 = Romance

('Dilan 1990', 'Pidi Baiq', 4, 2014, 20, 'Kisah romansa masa putih abu-abu di Bandung.', 210),

-- 5 = Fantasi

('Bumi', 'Tere Liye', 5, 2014, 10, 'Petualangan fantasi remaja di dunia paralel Klan Bulan.', 95),

-- 6 = Komik

('The Greatest Estate Developer', 'BK_Moon', 6, 2021, 25, 'Manhwa genre kingdom building tentang mahasiswa teknik sipil yang membangun wilayah.', 300);

Pembahasan Source Code :

Source code di atas menggunakan perintah INSERT INTO buku yang berfungsi untuk menambahkan data ke dalam tabel buku. Setelah nama tabel, dituliskan daftar kolom yang akan diisi, yaitu judul, penulis, kategori_id, tahun_terbit, stok, deskripsi, dan dilihat. Ini berarti setiap nilai yang dimasukkan harus mengikuti urutan kolom tersebut. Bagian VALUES digunakan untuk memasukkan data ke dalam tabel, di mana setiap pasangan tanda kurung () merepresentasikan satu baris data.

Di dalamnya, nilai dimasukkan sesuai urutan kolom, misalnya 'Basis Data Lanjut' untuk kolom judul, 'C.J. Date' untuk penulis, 2 untuk kategori_id, dan seterusnya. Tanda petik digunakan untuk tipe data teks (VARCHAR atau TEXT), sedangkan angka ditulis tanpa tanda petik karena merupakan tipe numerik. Penulisan beberapa data sekaligus dipisahkan dengan tanda koma antar baris, sehingga dalam satu perintah dapat menambahkan banyak data (multiple row insert). Hal ini lebih efisien dibandingkan menulis perintah INSERT satu per satu.

Komentar seperti -- 1 = Fiksi, 2 = Non Fiksi tidak akan dieksekusi oleh SQL, melainkan hanya sebagai catatan untuk menjelaskan arti dari nilai kategori_id yang digunakan. Selain itu, kolom id tidak dimasukkan dalam perintah karena sudah menggunakan AUTO_INCREMENT, sehingga nilainya akan dibuat otomatis oleh sistem setiap kali data baru ditambahkan. Dengan struktur ini, setiap baris data yang dimasukkan akan menjadi satu record baru dalam tabel buku sesuai dengan urutan dan tipe data yang telah ditentukan.

Printscreen Output:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	kategori_id	judul	penulis	tahun_terbit	stok	cover_url	deskripsi	dilihat
1	2	Basis Data Lanjut	C.J. Date	2023	5	NULL	Teori mendalam tentang SQL dan optimasi query.	45
2	2	Filosofi Teras	Henry Manampiring	2019	12	NULL	Penerapan Stoikisme dalam kehidupan modern.	88
3	1	Laskar Pelangi	Andrea Hirata	2005	8	NULL	Kisah perjuangan 10 anak di Belitong.	30
4	2	Python for AI	Guido Van Rossum	2024	15	NULL	Dasar pemrograman Python untuk AI.	120
5	3	Danur: Gerbang Dialog	Risa Saraswati	2011	7	NULL	Kisah persahabatan anak manusia dengan entit...	55
6	4	Dilan 1990	Pidi Baiq	2014	20	NULL	Kisah romansa masa putih abu-abu di Bandung.	210
7	5	Bumi	Tere Liye	2014	10	NULL	Petualangan fantasi remaja di dunia paralel Klan...	95
8	6	The Greatest Estate Developer	BK_Moon	2021	25	NULL	Manhwa genre kingdom building tentang mahasi...	300
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Kelompok 3

Gambar 2.20 Tabel Buku

Penjelasan Output:

Gambar di atas menampilkan isi data dari tabel buku pada database perpustakaan_digital. Tabel ini berfungsi sebagai katalog pusat yang menyimpan seluruh informasi mendetail mengenai koleksi buku yang tersedia di perpustakaan. Dalam tampilan tersebut, terdapat kolom id sebagai kunci utama, serta kategori_id yang berfungsi sebagai *foreign key* untuk menghubungkan buku dengan kategori tertentu (seperti Fiksi, Non-Fiksi, atau Horor).

Setiap baris data menyajikan informasi lengkap mulai dari judul, penulis, tahun_terbit, hingga jumlah stok yang tersedia untuk dipinjam. Terdapat pula kolom cover_url untuk menyimpan lokasi gambar sampul, deskripsi untuk ringkasan isi buku, serta kolom dilihat yang mencatat frekuensi interaksi pengguna terhadap judul buku tersebut. Data ini sangat penting bagi sistem untuk mengelola inventaris buku dan memberikan informasi yang akurat kepada anggota perpustakaan saat melakukan pencarian atau peminjaman.

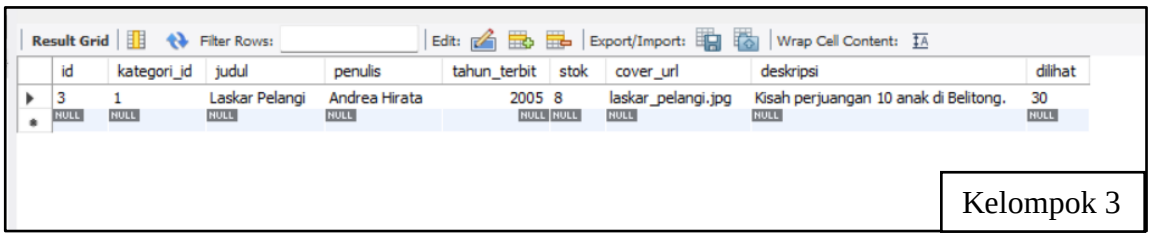
Source code:

```
UPDATE buku SET cover_url = 'laskar_pelangi.jpg' WHERE id = 3;
```

Pembahasan Source Code :

Perintah UPDATE buku SET cover_url = 'laskar_pelangi.jpg' WHERE id = 3; digunakan untuk memperbarui data pada tabel buku. Bagian SET cover_url = 'laskar_pelangi.jpg' menunjukkan bahwa kolom cover_url akan diisi atau diubah dengan nilai tersebut. Kondisi WHERE id = 3 berfungsi untuk menentukan data mana yang akan diubah, yaitu hanya data buku dengan id bernilai 3. Tanpa kondisi WHERE, semua data dalam tabel buku akan ikut terupdate. Dengan perintah ini, hanya satu record yang diperbarui, yaitu menambahkan atau mengganti URL cover pada buku tersebut.

Printscreen Output:



id	kategori_id	judul	penulis	tahun_terbit	stok	cover_url	deskripsi	dilihat
3	1	Laskar Pelangi	Andrea Hirata	2005	8	laskar_pelangi.jpg	Kisah perjuangan 10 anak di Belitong.	30
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Kelompok 3

Gambar 2.21 Output Update Buku Dengan Id = 3

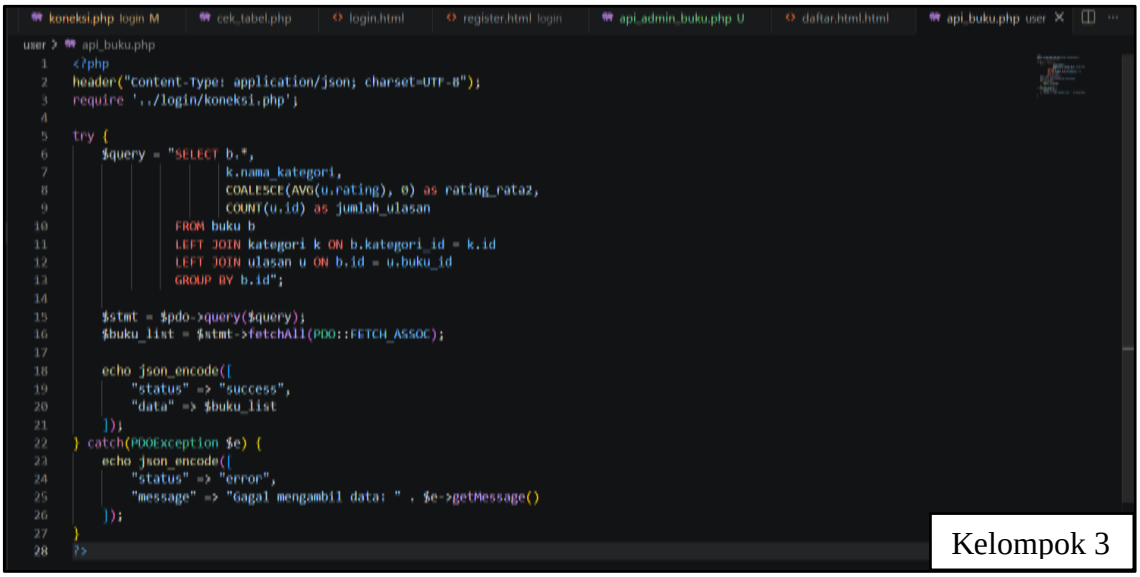
Penjelasan Output:

Gambar di atas menampilkan hasil eksekusi dari perintah UPDATE pada tabel buku. Perintah ini berfungsi untuk memperbarui atau mengubah data yang sudah ada di dalam database tanpa menghapus baris tersebut. Dalam kasus ini, pembaruan dilakukan pada

kolom cover_url untuk buku dengan id = 3 (Laskar Pelangi). Dapat dilihat pada result grid tersebut, kolom cover_url yang sebelumnya mungkin bernilai kosong (NULL), kini telah terisi dengan nama file 'laskar_pelangi.jpg'.

Proses update ini sangat penting dalam manajemen database perpustakaan untuk memastikan informasi buku tetap relevan dan lengkap, seperti saat admin mengunggah gambar sampul baru atau memperbaiki kesalahan penulisan pada deskripsi buku. Perubahan ini hanya berdampak pada baris yang memenuhi kondisi WHERE, sehingga data buku lainnya tetap terjaga konsistensinya.

Printscreen source code:



Gambar 2.22 Source Code Logika Pengambilan Katalog Buku (api_buku.php)

Source code:

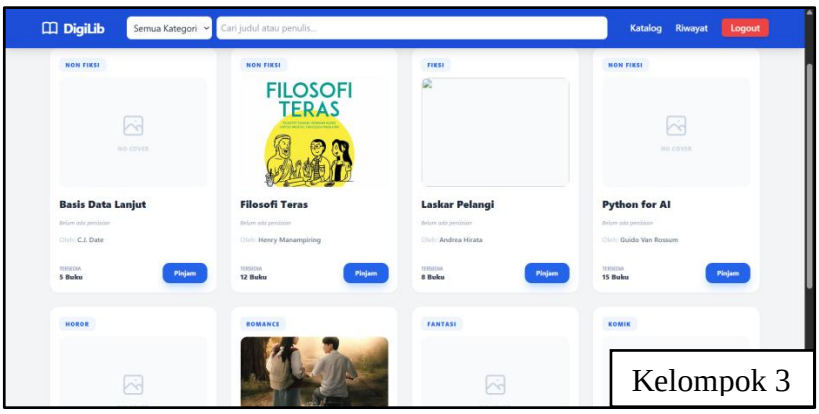
```
<?php
header("Content-Type: application/json; charset=UTF-8");
require '../login/koneksi.php';


try {
   $query = "SELECT b.*,
                  k.nama_kategori,
                  COALESCE(AVG(u.rating), 0) as rating_rata2,
                  COUNT(u.id) as jumlah_ulasan
            FROM buku b
            LEFT JOIN kategori k ON b.kategori_id = k.id
            LEFT JOIN ulasan u ON b.id = u.buku_id
            GROUP BY b.id";
```

```
    $stmt = $pdo->query($query);
    $buku_list = $stmt->fetchAll(PDO::FETCH_ASSOC);

    echo json_encode([
        "status" => "success",
        "data" => $buku_list
    ]);
} catch(PDOException $e) {
    echo json_encode([
        "status" => "error",
        "message" => "Gagal mengambil data: " . $e->getMessage()
    ]);
}
?>
```

Pembahasan Source Code :

Rangkaian gambar di atas memvisualisasikan fungsionalitas Katalog Buku pada sisi pengguna yang telah diintegrasikan dengan sistem penilaian (*rating*) secara dinamis. Pada arsitektur *backend*, aplikasi mengeksekusi kueri DQL relasional yang kompleks dengan melibatkan penggabungan tiga entitas sekaligus melalui klausa LEFT JOIN, yaitu tabel buku, kategori, dan ulasan. Struktur kueri SELECT b.*, k.nama_kategori, COALESCE(AVG(u.rating), 0) as rating_rata2, COUNT(u.id) as jumlah_ulasan FROM buku b LEFT JOIN kategori k ON b.kategori_id = k.id LEFT JOIN ulasan u ON b.id = u.buku_id GROUP BY b.id mendemonstrasikan pemanfaatan fungsi agregasi matematika AVG() untuk mengkalkulasi nilai rata-rata ulasan secara otomatis, di mana fungsi COALESCE() bertugas memanipulasi *output* agar mengembalikan angka 0 untuk buku yang belum memiliki ulasan, alih-alih merusak tampilan dengan nilai *null*. Himpunan data hasil pengelompokan (GROUP BY b.id) ini kemudian diekstraksi melalui $stmt->fetchAll(PDO::FETCH_ASSOC) dan direspons ke *browser* menggunakan echo json_encode(["status" => "success", "data" => $buku_list]). Pada sisi *frontend*, paket JSON tersebut diurai untuk merender antarmuka *card view* interaktif yang menampilkan sampul, metadata literatur, sisa stok, hingga visualisasi bintang ulasan secara *real-time*. Desain antarmuka yang transparan ini, dipadukan dengan ketersediaan tombol "Pinjam", sangat meningkatkan *User Experience* dengan memungkinkan anggota untuk membuat keputusan peminjaman berbasis rekam jejak penilaian dari pembaca sebelumnya.

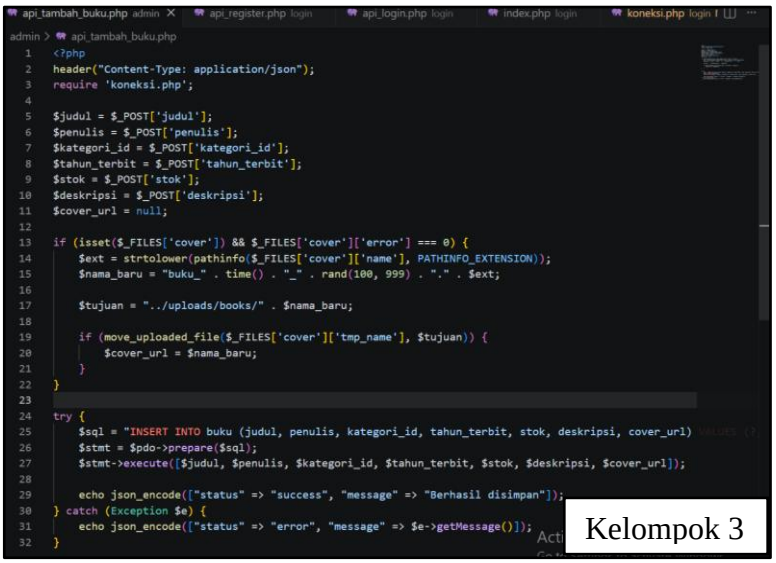
Tampilan SI:



Gambar 2.23 Antarmuka Katalog Buku Terintegrasi (Sisi Anggota)

Penjelasan Tampilan SI:

Gambar di atas merupakan wujud visual dari antarmuka Katalog Buku yang diakses oleh entitas dengan hak akses 'anggota'. Halaman ini secara langsung me-render keluaran data JSON yang dihasilkan oleh modul api_katalog_buku.php. Antarmuka dirancang menggunakan komponen *grid* berbasis *card view* yang secara dinamis menampilkan atribut literatur seperti label kategori, gambar sampul, judul buku, nama penulis, dan sisa stok secara *real-time*. Keberhasilan implementasi kueri LEFT JOIN dan fungsi agregasi COALESCE pada basis data terbukti secara visual pada elemen ini; buku-buku yang belum menerima umpan balik dari anggota akan secara otomatis menampilkan status "Belum ada penilaian" alih-alih merusak tata letak dengan nilai *null*. Selain itu, integrasi elemen navigasi atas berupa bilah pencarian (*search bar*) dinamis, *dropdown* filter kategori, serta ketersediaan tombol aksi "Pinjam" pada masing-masing kartu sangat memperkaya *User Experience* (UX), memungkinkan anggota untuk melakukan penelusuran dan sirkulasi literatur secara mandiri dan intuitif.



Gambar 2.24 Source Code Logika Tambah Koleksi Buku (api_tambah_buku.php)

Source code:

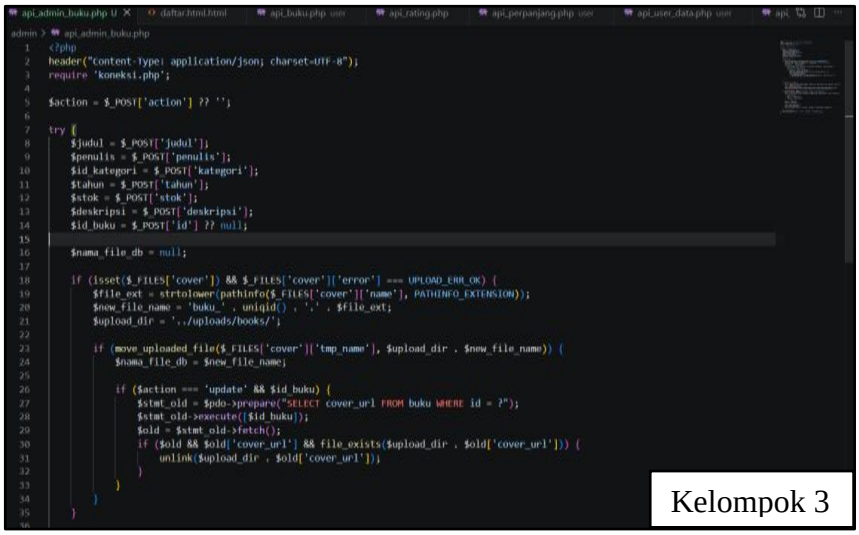
```
<?php
header("Content-Type: application/json");
require 'koneksi.php';

$judul = $_POST['judul'];
$penulis = $_POST['penulis'];
$kategori_id = $_POST['kategori_id'];
$tahun_terbit = $_POST['tahun_terbit'];
$stok = $_POST['stok'];
$deskripsi = $_POST['deskripsi'];
$cover_url = null;

if (isset($_FILES['cover']) && $_FILES['cover']['error'] === 0) {
    $ext = strtolower(pathinfo($_FILES['cover']['name'], PATHINFO_EXTENSION));
    $nama_baru = "buku_" . time() . "_" . rand(100, 999) . "." . $ext;

    $tujuan = "../uploads/books/" . $nama_baru;

    if (move_uploaded_file($_FILES['cover']['tmp_name'], $tujuan)) {
        $cover_url = $nama_baru;
    }
}

try {
    $sql = "INSERT INTO buku (judul, penulis, kategori_id, tahun_terbit, stok, deskripsi,
cover_url) VALUES (?, ?, ?, ?, ?, ?, ?)";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([$judul, $penulis, $kategori_id, $tahun_terbit, $stok, $deskripsi,
$cover_url]);

    echo json_encode(["status" => "success", "message" => "Berhasil disimpan"]);
} catch (Exception $e) {
    echo json_encode(["status" => "error", "message" => $e->getMessage()]);
}
```

Pembahasan Source Code :

Modul api_tambah_buku.php merupakan implementasi operasi *Create* pada sistem manajemen basis data yang berfungsi untuk menambahkan koleksi literatur baru ke dalam katalog perpustakaan digital. Skrip ini diawali dengan mengimpor konfigurasi server melalui require 'koneksi.php'; dan mendeklarasikan format *output* berupa JSON. Program kemudian menangkap berbagai atribut buku—seperti judul, penulis, kategori, dan stok—yang dikirimkan dari antarmuka formulir pengguna melalui variabel global $_POST. Selain data tekstual, sistem juga dilengkapi dengan algoritma penanganan berkas (file) untuk mengunggah gambar sampul buku. Berkas yang diunggah akan divalidasi kondisinya melalui $_FILES['cover']['error'] === 0, diubah namanya secara unik menggunakan kombinasi fungsi time() dan rand(), lalu dipindahkan ke direktori *server* menggunakan fungsi move_uploaded_file().

Proses penyimpanan inti ke dalam database dijalankan di dalam blok try { ... } menggunakan kueri DML (*Data Manipulation Language*) terstruktur, yaitu $sql = "INSERT INTO buku (judul, penulis, kategori_id, tahun_terbit, stok, deskripsi, cover_url) VALUES (?, ?, ?, ?, ?, ?, ?)";. Untuk menjaga integritas data dan mengamankan sistem dari celah *SQL Injection*, kueri tersebut tidak mengeksekusi data mentah secara langsung, melainkan menggunakan teknik *prepared statement* melalui perintah $stmt = $pdo->prepare($sql); yang disusul dengan pengikatan nilai pada $stmt->execute([...]);. Seluruh proses ini diakhiri dengan pengembalian respon ke klien menggunakan echo json_encode(["status" => "success", "message" => "Berhasil disimpan"]); agar halaman *frontend* Master Data Buku dapat memberikan notifikasi sukses kepada admin

Printscreen source code:



Gambar 2.25 Source Code Logika Update Buku (api_admin_buku)

Source code:

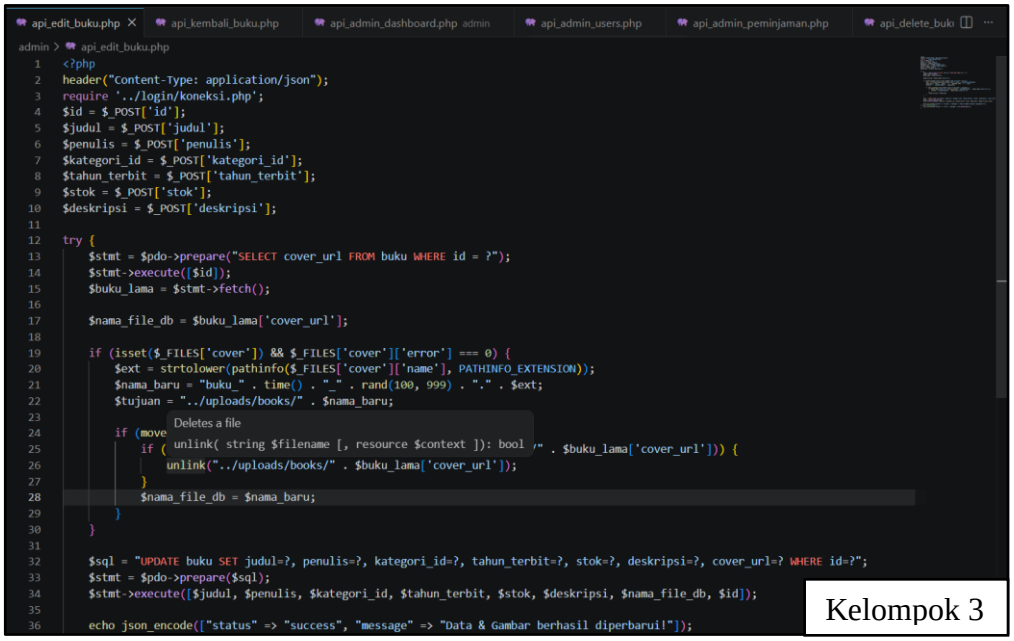
```
<?php
header("Content-Type: application/json; charset=UTF-8");
require 'koneksi.php';

$action = $_POST['action'] ?? '';

try {
    $judul = $_POST['judul'];
    $penulis = $_POST['penulis'];
    $id_kategori = $_POST['kategori'];
    $tahun = $_POST['tahun'];
    $stok = $_POST['stok'];
    $deskripsi = $_POST['deskripsi'];
    $id_buku = $_POST['id'] ?? null;

    $nama_file_db = null;

    if (isset($_FILES['cover']) && $_FILES['cover']['error'] === UPLOAD_ERR_OK) {
        $file_ext = strtolower(pathinfo($_FILES['cover']['name'],
PATHINFO_EXTENSION));
        $new_file_name = 'buku_' . uniqid() . '.' . $file_ext;
        $upload_dir = '../uploads/books/';

        if (move_uploaded_file($_FILES['cover']['tmp_name'], $upload_dir .
$new_file_name)) {
            $nama_file_db = $new_file_name;

            if ($action === 'update' && $id_buku) {
                $stmt_old = $pdo->prepare("SELECT cover_url FROM buku WHERE id = ?");
                $stmt_old->execute([$id_buku]);
                $old = $stmt_old->fetch();
                if ($old && $old['cover_url'] && file_exists($upload_dir . $old['cover_url'])) {
                    unlink($upload_dir . $old['cover_url']);
                }
```

```
            }
        }
    }

    if ($action === 'insert') {
        $sql = "INSERT INTO buku (judul, penulis, kategori_id, tahun_terbit, stok, deskripsi,
cover_url)
                VALUES (?, ?, ?, ?, ?, ?, ?)";
        $stmt = $pdo->prepare($sql);
        $stmt->execute([$judul, $penulis, $id_kategori, $tahun, $stok, $deskripsi,
$nama_file_db]);
        echo json_encode(["status" => "success", "message" => "Buku berhasil ditambah!"]);

    } elseif ($action === 'update') {
        $params = [$judul, $penulis, $id_kategori, $tahun, $stok, $deskripsi];
        $sql = "UPDATE buku SET judul=?, penulis=?, kategori_id=?, tahun_terbit=?,
stok=?, deskripsi=?";

        if ($nama_file_db) {
            $sql .= ", cover_url=?";
            $params[] = $nama_file_db;
        }

        $sql .= " WHERE id=?";
        $params[] = $id_buku;

        $stmt = $pdo->prepare($sql);
        $stmt->execute($params);
        echo json_encode(["status" => "success", "message" => "Buku berhasil diupdate!"]);
    }

} catch (Exception $e) {
    echo json_encode(["status" => "error", "message" => $e->getMessage()]);
}
?>
```

Pembahasan Source Code :

Modul pengelola buku ini menggunakan kondisi percabangan elseif ($action === 'update') untuk memproses pembaruan data literatur. Keunggulan utama pada skrip ini adalah implementasi kueri dinamis serta manajemen berkas yang efisien. Saat administrator mengunggah sampul buku yang baru, sistem tidak langsung menimpanya, melainkan mengeksekusi kueri penelusuran SELECT cover_url FROM buku WHERE id = ? terlebih dahulu. Jika file lama ditemukan di dalam direktori *server*, sistem akan menghapusnya secara fisik menggunakan fungsi unlink($upload_dir . $old['cover_url']) guna mencegah penumpukan berkas yang tidak terpakai. Selanjutnya, struktur dasar kueri pembaruan dirangkai di dalam variabel $sql = "UPDATE buku SET judul=?, penulis=?, kategori_id=?, tahun_terbit=?, stok=?, deskripsi=?". Jika terdapat berkas gambar baru, klausa tambahan, cover_url=? akan digabungkan ($sql .= ", cover_url=?") ke dalam string kueri sebelum diakhiri dengan klausa kondisional $sql .= " WHERE id=?" dan dieksekusi oleh PDO.

Printscreen source code:



Gambar 2.26 Source Code Update Koleksi Buku (api_edit_buku.php)

Source code:

```
<?php
header("Content-Type: application/json");
require '../login/koneksi.php';
$id = $_POST['id'];
$judul = $_POST['judul'];
$penulis = $_POST['penulis'];
```

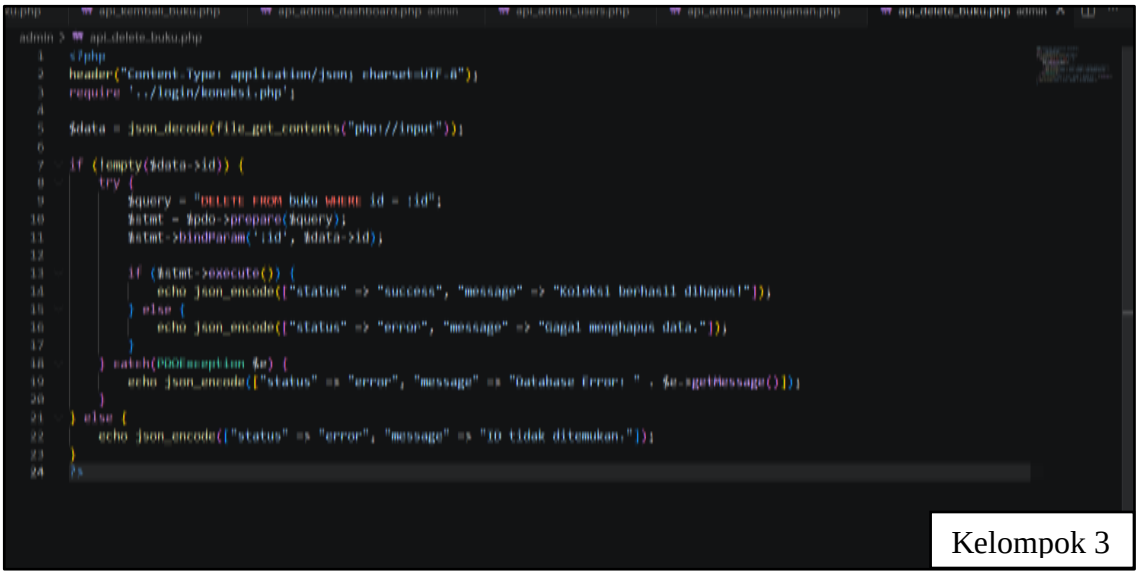
```
$kategori_id = $_POST['kategori_id'];
$tahun_terbit = $_POST['tahun_terbit'];
$stok = $_POST['stok'];
$deskripsi = $_POST['deskripsi'];

try {
    $stmt = $pdo->prepare("SELECT cover_url FROM buku WHERE id = ?");
    $stmt->execute([$id]);
    $buku_lama = $stmt->fetch();

    $nama_file_db = $buku_lama['cover_url'];

    if (isset($_FILES['cover']) && $_FILES['cover']['error'] === 0) {
        $ext = strtolower(pathinfo($_FILES['cover']['name'], PATHINFO_EXTENSION));
        $nama_baru = "buku_" . time() . "_" . rand(100, 999) . "." . $ext;
        $tujuan = "../uploads/books/" . $nama_baru;

        if (move_uploaded_file($_FILES['cover']['tmp_name'], $tujuan)) {
            if ($buku_lama['cover_url'] && file_exists("../uploads/books/" .
$buku_lama['cover_url'])) {
                unlink("../uploads/books/" . $buku_lama['cover_url']);
            }
            $nama_file_db = $nama_baru;
        }
    }

    $sql = "UPDATE buku SET judul=?, penulis=?, kategori_id=?, tahun_terbit=?, stok=?,
deskripsi=?, cover_url=? WHERE id=?";
    $stmt = $pdo->prepare($sql);
    $stmt->execute([$judul, $penulis, $kategori_id, $tahun_terbit, $stok, $deskripsi,
$nama_file_db, $id]);

    echo json_encode(["status" => "success", "message" => "Data & Gambar berhasil
diperbarui!"]);
} catch (Exception $e) {
```

```
    echo json_encode(["status" => "error", "message" => $e->getMessage()]);
}
```

Pembahasan Source Code :

Modul api_edit_buku.php merupakan implementasi dari operasi DML tingkat lanjut yang bertujuan untuk memodifikasi atau memperbarui (*Update*) informasi literatur yang sudah ada di dalam katalog perpustakaan. Proses ini dimulai dengan menangkap seluruh data inputan dari antarmuka admin, termasuk id buku sebagai kata kunci pencarian. Sebelum mengubah data teks, sistem menjalankan kueri SELECT cover_url FROM buku WHERE id = ? untuk mengambil informasi sampul lama. Jika admin mengunggah sampul buku yang baru, sistem akan menyimpan berkas baru tersebut dan langsung mengeksekusi fungsi bawaan PHP yaitu unlink() guna menghapus berkas sampul fisik yang lama dari direktori *server*, sehingga kapasitas penyimpanan tetap efisien. Operasi inti pada modul ini dijalankan melalui kueri SQL UPDATE buku SET judul=?, penulis=?, kategori_id=?, tahun_terbit=?, stok=?, deskripsi=?, cover_url=? WHERE id=?. Penggunaan klausa WHERE id=? sangat vital dalam struktur modifikasi ini untuk memastikan bahwa perubahan data hanya diaplikasikan secara eksklusif pada satu baris *record* buku yang spesifik. Seluruh parameter diikat menggunakan metode *prepared statement* guna mencegah modifikasi tidak sah melalui celah *SQL Injection*.

Printscreen source code:



Gambar 2.27 Source Code Menghapus Koleksi Buku (api_delete_buku.php)

Source code:

```
<?php
header("Content-Type: application/json; charset=UTF-8");
require '../login/koneksi.php';
```

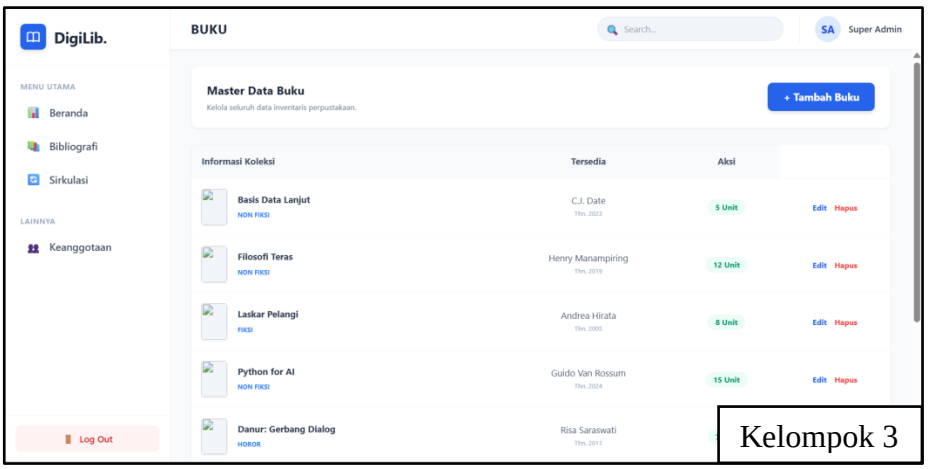
```
$data = json_decode(file_get_contents("php://input"));

if (!empty($data->id)) {
    try {
        $query = "DELETE FROM buku WHERE id = :id";
        $stmt = $pdo->prepare($query);
        $stmt->bindParam(':id', $data->id);

        if ($stmt->execute()) {
            echo  json_encode(["status"  =>  "success",  "message"  =>  "Koleksi  berhasil
dihapus!"]);
        } else {
            echo json_encode(["status" => "error", "message" => "Gagal menghapus data."]);
        }
    } catch(PDOException $e) {
        echo  json_encode(["status"  =>  "error",  "message"  =>  "Database  Error:  "  .  $e-
>getMessage()]);
    }
} else {
    echo json_encode(["status" => "error", "message" => "ID tidak ditemukan."]);
}
?>
```

Pembahasan Source Code :

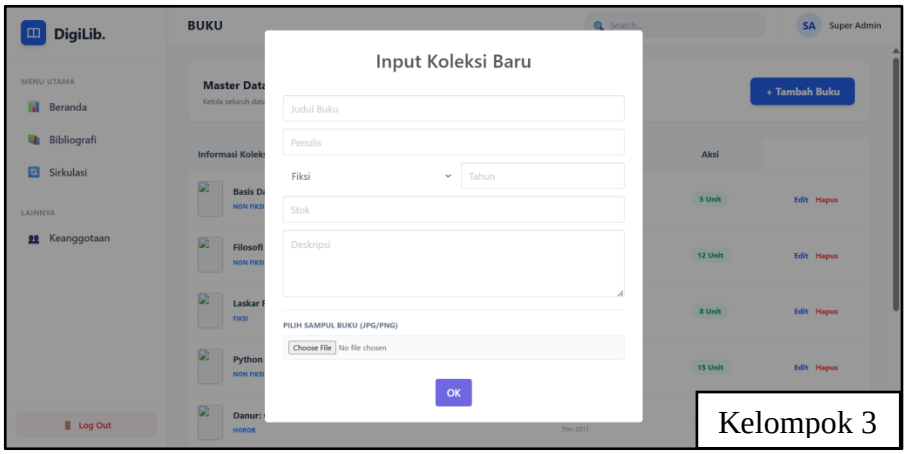
Modul api_delete_buku.php berfungsi untuk mengeksekusi operasi DML (*Data Manipulation Language*) tahap akhir, yaitu penghapusan *record* koleksi buku dari basis data. Skrip ini menerima input parameter berformat JSON yang berisi atribut ID buku. Operasi utama dijalankan menggunakan kueri SQL DELETE FROM buku WHERE id = :id. Penggunaan klausa kondisional WHERE yang dikombinasikan dengan teknik *prepared statement* PDO (bindParam) merupakan langkah pengamanan prosedural yang esensial untuk melindungi database dari potensi ancaman siber seperti injeksi SQL, sekaligus memastikan sistem hanya menghapus baris data yang benar-benar dituju secara presisi. Lebih lanjut, efisiensi modul ini sangat didukung oleh rancangan arsitektur basis data yang telah mengimplementasikan *constraint Foreign Key* dengan aturan ON DELETE CASCADE pada tabel peminjaman dan tabel ulasan. Berkat integrasi relasional tersebut,

saat perintah hapus ini dieksekusi pada tabel master buku, sistem (MySQL) akan secara otomatis membersihkan seluruh riwayat sirkulasi dan data komentar yang terikat dengan ID buku tersebut. Mekanisme ini menjamin terwujudnya konsistensi data (*referential integrity*) tanpa mengharuskan pengembang menuliskan perintah kueri penghapusan berganda secara manual.

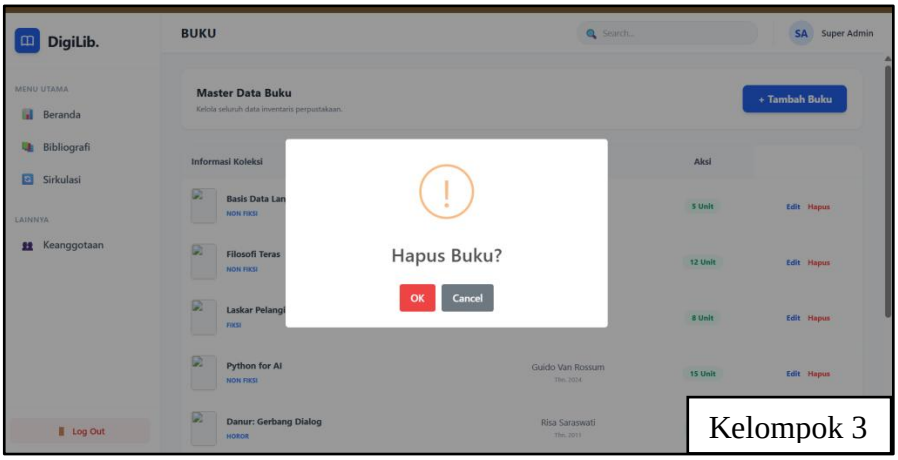
Tampilan SI:



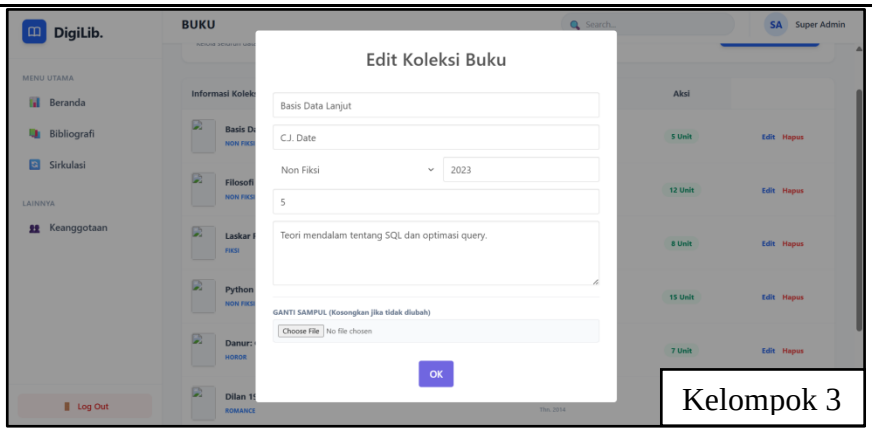
Gambar 2.28 Antarmuka Input Buku (Sisi Admin)



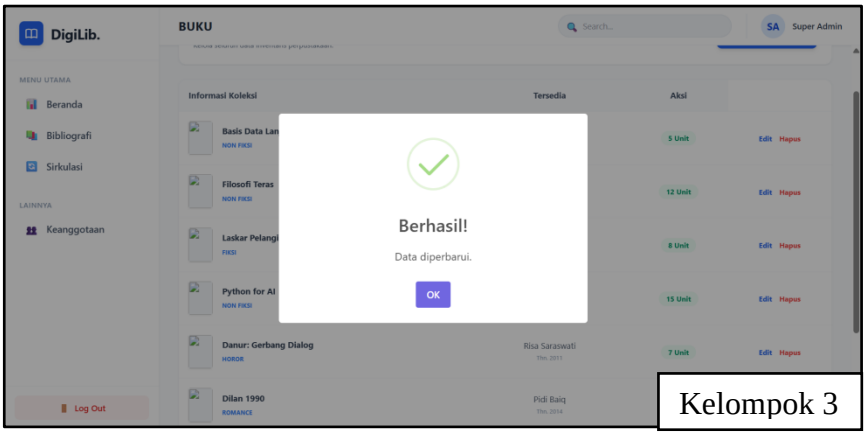
Gambar 2.29 Antarmuka Input Buku (Sisi Admin)



Gambar 2.30 Verifikasi Penghapusan pada Tabel Buku (Sisi Admin)



Gambar 2.31 Edit Buku (Sisi Admin)



Gambar 2.32 Notifikasi Keberhasilan Pengeditan Buku (Sisi Admin)

Penjelasan Tampilan SI:

Rangkaian antarmuka pada sistem DigiLib memfasilitasi administrator dalam melakukan pengelolaan master data buku melalui siklus CRUD yang terintegrasi secara dinamis antara *frontend* dan *backend*. Proses pengelolaan koleksi diawali dengan pengisian data bibliografi melalui formulir Tambah Buku, hingga mekanisme pembaruan informasi pada formulir Edit Buku yang secara otomatis melakukan *binding* data lama dari database ke dalam kolom *input*. Untuk menjaga integritas data pada fitur penghapusan, sistem menerapkan protokol keamanan berlapis yang dimulai dengan jendela konfirmasi "Hapus Buku" untuk memvalidasi aksi administrator. Pada lapisan *backend*, modul api_delete_buku.php akan memproses permintaan dengan mengeksekusi perintah SQL DELETE FROM buku WHERE id = :id menggunakan *prepared statements* melalui fungsi $stmt->bindParam(':id', $data->id) dan $stmt->execute() guna meniadakan risiko serangan injeksi. Respon dari server kemudian dipetakan kembali ke layar pengguna dalam bentuk notifikasi status, baik berupa konfirmasi sukses "Koleksi berhasil dihapus!" maupun peringatan galat jika terjadi kendala teknis pada level basis data. Akhir dari seluruh rangkaian transaksi ini dapat divalidasi secara absolut melalui tabel buku pada MySQL, di

mana baris data yang menjadi objek manipulasi telah diperbarui atau dihapus secara permanen sesuai dengan instruksi yang dikirimkan.

Printscreen source code:

```
CREATE TABLE peminjaman (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    buku_id INT,
    tanggal_pinjam DATE NOT NULL,
    tanggal_kembali DATE,
    status ENUM('dipinjam', 'dikembalikan') DEFAULT 'dipinjam',
    jumlah_perpanjang INT DEFAULT 0,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (buku_id) REFERENCES buku(id) ON DELETE CASCADE
);
```

Kelompok 3

Gambar 2.33 Membuat Tabel Peminjaman

Source code:

```
CREATE TABLE peminjaman (
   id INT AUTO_INCREMENT PRIMARY KEY,
   user_id INT,
   buku_id INT,
   tanggal_pinjam DATE NOT NULL,
   tanggal_kembali DATE,
   status ENUM('dipinjam', 'dikembalikan') DEFAULT 'dipinjam',
   jumlah_perpanjang INT DEFAULT 0,
   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
   FOREIGN KEY (buku_id) REFERENCES buku(id) ON DELETE CASCADE
);
```

Pembahasan Source Code :

Perintah CREATE TABLE peminjaman digunakan untuk membuat tabel peminjaman yang berfungsi menyimpan data transaksi peminjaman buku. Kolom id bertipe INT dengan AUTO_INCREMENT dan dijadikan PRIMARY KEY, sehingga setiap data peminjaman memiliki identitas unik yang dibuat otomatis. Kolom user_id dan buku_id bertipe INT berfungsi sebagai foreign key yang menghubungkan tabel peminjaman dengan tabel users dan buku. Kolom tanggal_pinjam bertipe DATE dan wajib diisi (NOT NULL), sedangkan tanggal_kembali juga bertipe DATE tetapi boleh kosong karena buku belum tentu langsung dikembalikan. Kolom status menggunakan tipe ENUM dengan pilihan nilai 'dipinjam' atau 'dikembalikan', dan memiliki nilai default

'dipinjam'. Kolom jumlah_perpanjang bertipe INT dengan nilai default 0 untuk mencatat berapa kali peminjaman diperpanjang. Selain itu, terdapat constraint FOREIGN KEY dengan aturan ON DELETE CASCADE, yang berarti jika data user atau buku dihapus, maka data peminjaman yang terkait juga akan ikut terhapus secara otomatis.

Struktur tabel ini mengelola alur sirkulasi buku secara sistematis dan otomatis di dalam sistem perpustakaan digital. Dengan adanya kolom status dan tanggal_kembali, sistem dapat melakukan validasi data secara akurat untuk membedakan antara buku yang masih berada di tangan peminjam dengan buku yang sudah tersedia kembali di rak. Selain itu, penggunaan atribut ON DELETE CASCADE pada relasi *foreign key* menjamin konsistensi data (*referential integrity*), sehingga tidak akan ada data transaksi yang tidak terhubung atau tidak valid jika data pengguna atau data buku dihapus dari sistem. Hal ini memudahkan admin dalam memantau statistik aktivitas peminjaman setiap bulannya tanpa perlu melakukan pembersihan data secara manual..

Printscreen source code:

```
INSERT INTO peminjaman (user_id, buku_id, tanggal_pinjam, status) VALUES
(2, 1, '2026-04-10', 'dipinjam'),
(3, 2, '2026-04-12', 'dipinjam');

INSERT INTO peminjaman (user_id, buku_id, tanggal_pinjam, tanggal_kembali, status) VALUES
(2, 3, '2026-01-10', '2026-01-15', 'dikembalikan'),
(3, 5, '2026-01-18', '2026-01-22', 'dikembalikan'),
(4, 8, '2026-01-25', '2026-01-30', 'dikembalikan'),

(2, 1, '2026-02-05', '2026-02-10', 'dikembalikan'),
(3, 2, '2026-02-12', '2026-02-18', 'dikembalikan'),
(4, 6, '2026-02-15', '2026-02-20', 'dikembalikan'),
(2, 7, '2026-02-20', '2026-02-25', 'dikembalikan'),

(3, 4, '2026-03-02', '2026-03-08', 'dikembalikan'),
(4, 5, '2026-03-08', '2026-03-12', 'dikembalikan'),
(2, 6, '2026-03-15', '2026-03-20', 'dikembalikan'),
(3, 7, '2026-03-22', '2026-03-27', 'dikembalikan'),
(4, 1, '2026-03-28', '2026-04-02', 'dikembalikan');
```

Kelompok 3

Gambar 2.34 Tambah Data Peminjaman

Source code:

INSERT INTO peminjaman (user_id, buku_id, tanggal_pinjam, status) VALUES
(2, 1, '2026-04-10', 'dipinjam'),
(3, 2, '2026-04-12', 'dipinjam');

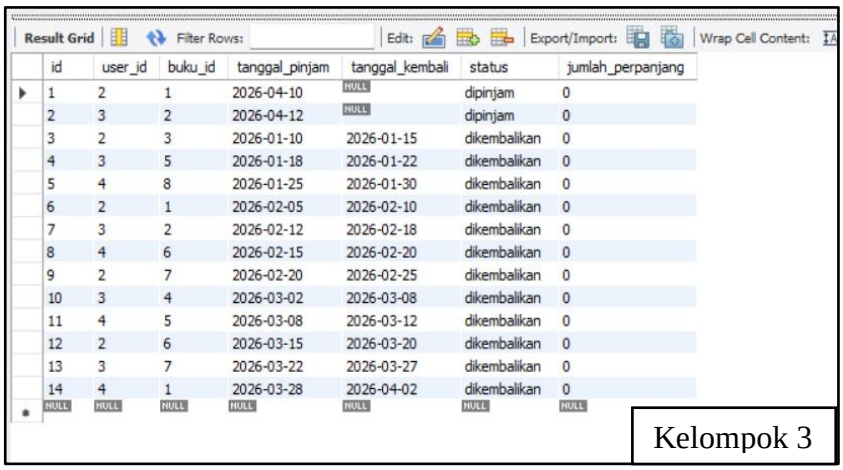
INSERT INTO peminjaman (user_id, buku_id, tanggal_pinjam, tanggal_kembali, status) VALUES
(2, 3, '2026-01-10', '2026-01-15', 'dikembalikan'),
(3, 5, '2026-01-18', '2026-01-22', 'dikembalikan'),
(4, 8, '2026-01-25', '2026-01-30', 'dikembalikan'),

```
(2, 1, '2026-02-05', '2026-02-10', 'dikembalikan'),
(3, 2, '2026-02-12', '2026-02-18', 'dikembalikan'),
(4, 6, '2026-02-15', '2026-02-20', 'dikembalikan'),
(2, 7, '2026-02-20', '2026-02-25', 'dikembalikan'),

(3, 4, '2026-03-02', '2026-03-08', 'dikembalikan'),
(4, 5, '2026-03-08', '2026-03-12', 'dikembalikan'),
(2, 6, '2026-03-15', '2026-03-20', 'dikembalikan'),
(3, 7, '2026-03-22', '2026-03-27', 'dikembalikan'),
(4, 1, '2026-03-28', '2026-04-02', 'dikembalikan');
```

Pembahasan Source Code :

Perintah INSERT INTO peminjaman digunakan untuk menambahkan data ke tabel peminjaman. Pada bagian pertama, data dimasukkan ke kolom user_id, buku_id, tanggal_pinjam, dan status, tanpa mengisi tanggal_kembali karena buku masih dalam status 'dipinjam'. Hal ini menunjukkan bahwa transaksi peminjaman masih berlangsung. Pada bagian kedua, data dimasukkan dengan tambahan kolom tanggal_kembali, yang menandakan bahwa buku sudah dikembalikan dengan status 'dikembalikan'. Setiap pasangan tanda kurung () merepresentasikan satu record, dan beberapa data dimasukkan sekaligus untuk efisiensi penulisan. Nilai user_id dan buku_id merupakan foreign key yang menghubungkan data ke tabel users dan buku, sehingga dapat diketahui siapa yang meminjam dan buku apa yang dipinjam. Sementara itu, tanggal_pinjam dan tanggal_kembali menunjukkan periode peminjaman. Dengan struktur ini, tabel peminjaman dapat menyimpan riwayat transaksi, baik yang masih aktif maupun yang sudah selesai.

Printscreen Output:



id	user_id	buku_id	tanggal_pinjam	tanggal_kembali	status	jumlah_perpanjang
1	2	1	2026-04-10	NULL	dipinjam	0
2	3	2	2026-04-12	NULL	dipinjam	0
3	2	3	2026-01-10	2026-01-15	dikembalikan	0
4	3	5	2026-01-18	2026-01-22	dikembalikan	0
5	4	8	2026-01-25	2026-01-30	dikembalikan	0
6	2	1	2026-02-05	2026-02-10	dikembalikan	0
7	3	2	2026-02-12	2026-02-18	dikembalikan	0
8	4	6	2026-02-15	2026-02-20	dikembalikan	0
9	2	7	2026-02-20	2026-02-25	dikembalikan	0
10	3	4	2026-03-02	2026-03-08	dikembalikan	0
11	4	5	2026-03-08	2026-03-12	dikembalikan	0
12	2	6	2026-03-15	2026-03-20	dikembalikan	0
13	3	7	2026-03-22	2026-03-27	dikembalikan	0
14	4	1	2026-03-28	2026-04-02	dikembalikan	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Kelompok 3

Gambar 2.35 Tabel Peminjaman

Penjelasan Output:

Gambar di atas menampilkan isi data dari tabel peminjaman pada database perpustakaan_digital. Tabel ini berfungsi untuk mencatat setiap transaksi sirkulasi buku yang dilakukan oleh anggota perpustakaan. Dalam tampilan tersebut, terdapat kolom id sebagai identitas unik transaksi, serta kolom user_id dan buku_id yang berfungsi menghubungkan data peminjaman dengan data pengguna dan buku yang bersangkutan.

Data tersebut menyajikan informasi operasional yang lengkap, mulai dari tanggal_pinjam, tanggal_kembali, hingga status yang menunjukkan apakah buku tersebut masih 'dipinjam' atau sudah 'dikembalikan'. Selain itu, terdapat kolom jumlah_perpanjang untuk melacak berapa kali masa pinjam buku tersebut telah diperbarui oleh pengguna. Tabel ini memiliki peran vital dalam mengontrol ketersediaan stok buku secara *real-time* dan memantau riwayat aktivitas peminjaman di dalam sistem.

Printscreen source code:

Gambar 2.36 Source Code Logika Peminjaman Buku (api_pinjam.php)

Source code:

```
<?php
header("Content-Type: application/json; charset=UTF-8");
require '../login/koneksi.php';

$data = json_decode(file_get_contents("php://input"));

if (!empty($data->buku_id) && !empty($data->user_id) && !empty($data->tanggal_kembali)) {
    try {
        $cek = $pdo->prepare("SELECT stok FROM buku WHERE id = :id");
```

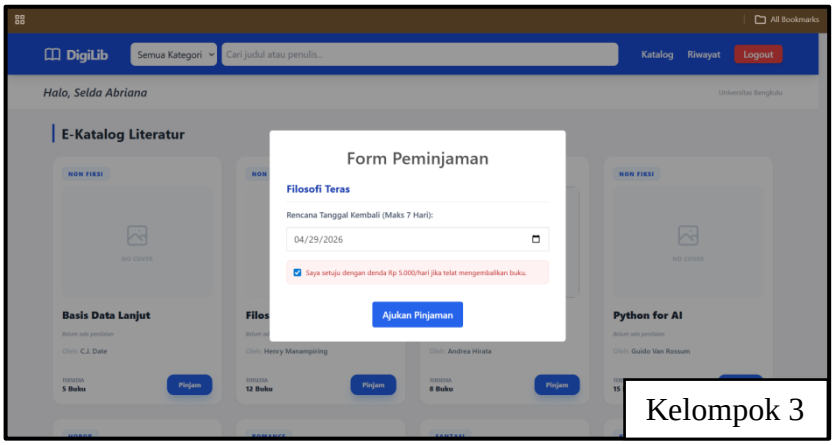
```
        $cek->bindParam(':id', $data->buku_id);
        $cek->execute();
        $buku = $cek->fetch(PDO::FETCH_ASSOC);

        if ($buku['stok'] > 0) {
            $update = $pdo->prepare("UPDATE buku SET stok = stok - 1 WHERE id = :id");
            $update->bindParam(':id', $data->buku_id);
            $update->execute();

            $insert = $pdo->prepare("INSERT INTO peminjaman (user_id, buku_id,
tanggal_pinjam, tanggal_kembali, status) VALUES (:uid, :bid, CURDATE(), :tgl_kembali,
'dipinjam')");
            $insert->bindParam(':uid', $data->user_id);
            $insert->bindParam(':bid', $data->buku_id);
            $insert->bindParam(':tgl_kembali', $data->tanggal_kembali);
            $insert->execute();

            echo json_encode(["status" => "success", "message" => "Buku berhasil dipinjam!
Jangan lupa kembalikan tepat waktu."]);
        } else {
            echo json_encode(["status" => "error", "message" => "Maaf, stok buku ini sedang
habis!"]);
        }
    } catch(PDOException $e) {
        echo json_encode(["status" => "error", "message" => "Error Database: " . $e-
>getMessage()]);
    }
} else {
    echo json_encode(["status" => "error", "message" => "Data tidak lengkap atau tanggal
belum dipilih!"]);
}
?>
```

Pembahasan Source Code :

Modul api_pinjam.php merupakan komponen krusial yang mengelola proses "Barang Keluar" atau sirkulasi peminjaman secara mandiri oleh anggota dari antarmuka katalog.

Skrip ini menangkap *payload* JSON yang berisi identitas buku, pengguna, dan batas waktu pengembalian melalui fungsi $data = json_decode(file_get_contents("php://input")). Sebelum merekam transaksi, sistem mengimplementasikan validasi logis dengan mengeksekusi kueri DQL SELECT stok FROM buku WHERE id = :id guna memastikan ketersediaan fisik literatur di rak perpustakaan. Apabila kondisi if ($buku['stok'] > 0) terpenuhi, sistem akan menjalankan dua instruksi DML secara simultan. Instruksi pertama adalah UPDATE buku SET stok = stok - 1 WHERE id = :id yang bertugas memotong jumlah ketersediaan buku secara *real-time*. Setelah stok berhasil disesuaikan, sistem langsung merekam riwayat sirkulasi melalui instruksi INSERT INTO peminjaman (user_id, buku_id, tanggal_pinjam, tanggal_kembali, status) VALUES (:uid, :bid, CURDATE(), :tgl_kembali, 'dipinjam'). Penerapan fungsi temporal CURDATE() memastikan waktu awal peminjaman tercatat secara presisi berdasarkan waktu *server*, sementara seluruh parameter dikonfigurasi menggunakan metode *prepared statements* (bindParam) untuk menangkis ancaman *SQL Injection*.

Tampilan SI:



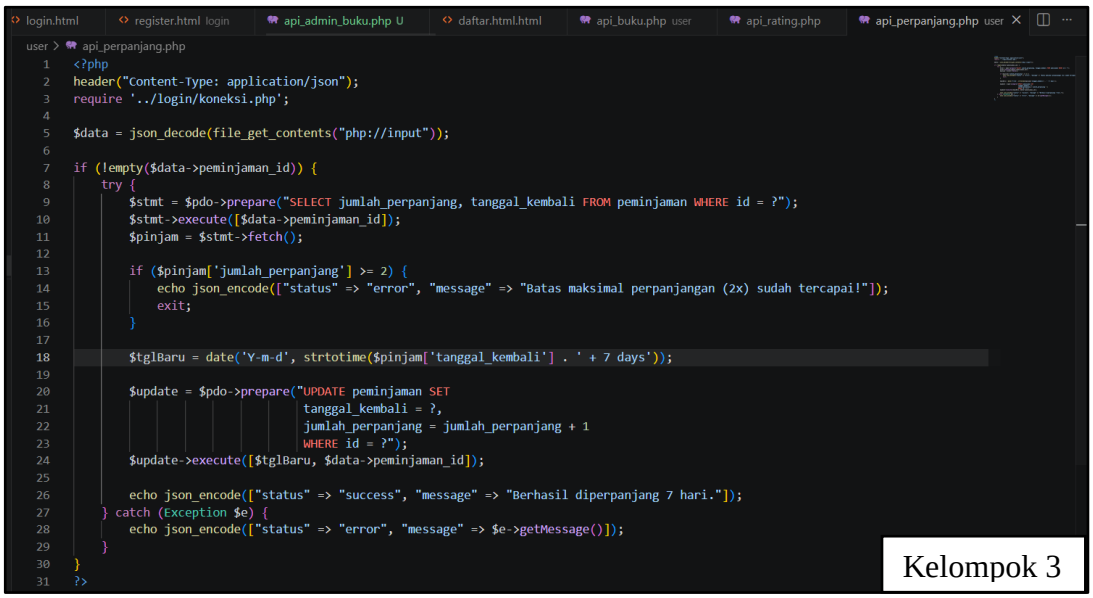
Gambar 2.37 Antarmuka Formulir Peminjaman Buku (Sisi User)

Penjelasan Tampilan SI:

Gambar di atas memvisualisasikan antarmuka *frontend* interaktif yang menjembatani interaksi anggota dengan modul *backend* sirkulasi. Ketika anggota menekan tombol "Pinjam" pada salah satu literatur di E-Katalog, sistem merespons dengan menampilkan *modal overlay* "Form Peminjaman" secara dinamis. Pada antarmuka ini, sistem mewajibkan anggota untuk menentukan tenggat waktu peminjaman melalui komponen *date picker* pada kolom "Rencana Tanggal Kembali", yang secara logika bisnis dibatasi maksimal 7 hari dari tanggal transaksi. Selain elemen input waktu, sistem juga menerapkan fitur kontrol validasi berupa *checkbox* persetujuan (*Term and Condition*) yang mengikat pengguna pada aturan denda keterlambatan sebesar Rp 5.000/hari. Implementasi *checkbox* ini merupakan strategi mitigasi risiko yang sangat efektif untuk mendisiplinkan anggota.

Setelah formulir divalidasi pada sisi klien (*client-side*), data tersebut kemudian dikirimkan dalam format JSON menuju modul api_pinjam.php untuk dieksekusi dan dicatat ke dalam basis data.

Printscreen source code:



Gambar 2.38 Source Code Logika Perpanjangan Peminjaman (api_perpanjang.php)

Source code:

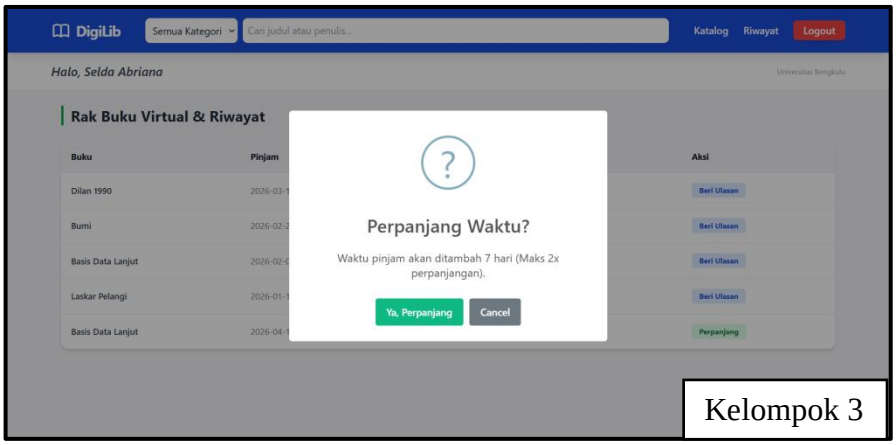
```
<?php
header("Content-Type: application/json");
require '../login/koneksi.php';

$data = json_decode(file_get_contents("php://input"));

if (!empty($data->peminjaman_id)) {
    try {
        $stmt = $pdo->prepare("SELECT jumlah_perpanjang, tanggal_kembali FROM peminjaman WHERE id = ?");
        $stmt->execute([$data->peminjaman_id]);
        $pinjam = $stmt->fetch();

        if ($pinjam['jumlah_perpanjang'] >= 2) {
            echo json_encode(["status" => "error", "message" => "Batas maksimal perpanjangan (2x) sudah tercapai!"]);
            exit;
        }
```

```
        $tglBaru = date('Y-m-d', strtotime($pinjam['tanggal_kembali'] . ' + 7 days'));

        $update = $pdo->prepare("UPDATE peminjaman SET
                                tanggal_kembali = ?,
                                jumlah_perpanjang = jumlah_perpanjang + 1
                                WHERE id = ?");
        $update->execute([$tglBaru, $data->peminjaman_id]);

        echo json_encode(["status" => "success", "message" => "Berhasil diperpanjang 7 hari."]);
    } catch (Exception $e) {
        echo json_encode(["status" => "error", "message" => $e->getMessage()]);
    }
}
?>
```

Pembahasan Source Code :

Modul api_perpanjang.php merupakan fungsionalitas tambahan pada sisi anggota yang memfasilitasi perpanjangan durasi sirkulasi buku secara mandiri. Skrip ini memproses parameter identitas transaksi melalui instruksi `$data = json_decode(file_get_contents("php://input"))`. Sebelum mengizinkan pembaruan data, sistem menjalankan fungsi validasi dengan mengeksekusi kueri DQL `SELECT jumlah_perpanjang, tanggal_kembali FROM peminjaman WHERE id = ?`. Aplikasi menerapkan aturan bisnis (*business logic*) yang ketat menggunakan struktur kontrol `if ($pinjam['jumlah_perpanjang'] >= 2)`; di mana jika terdeteksi bahwa anggota telah melakukan perpanjangan sebanyak dua kali, maka permintaan akan langsung ditolak guna mencegah monopoli koleksi literatur. Apabila lolos tahap validasi, sistem akan mengkalkulasi tenggat waktu pengembalian yang baru menggunakan fungsi manipulasi waktu bawaan PHP, yaitu `$tglBaru = date('Y-m-d', strtotime($pinjam['tanggal_kembali'] . ' + 7 days'))`, yang secara matematis menambahkan durasi 7 hari dari batas waktu sebelumnya. Modifikasi ini kemudian disimpan ke dalam basis data melalui kueri pembaruan `UPDATE peminjaman SET tanggal_kembali = ?, jumlah_perpanjang = jumlah_perpanjang + 1 WHERE id = ?`, yang dieksekusi secara aman menggunakan parameterisasi `$update->execute([$tglBaru, $data->peminjaman_id])` untuk memperbarui tenggat waktu sekaligus menambahkan *counter* riwayat perpanjangan buku tersebut.

Tampilan SI:

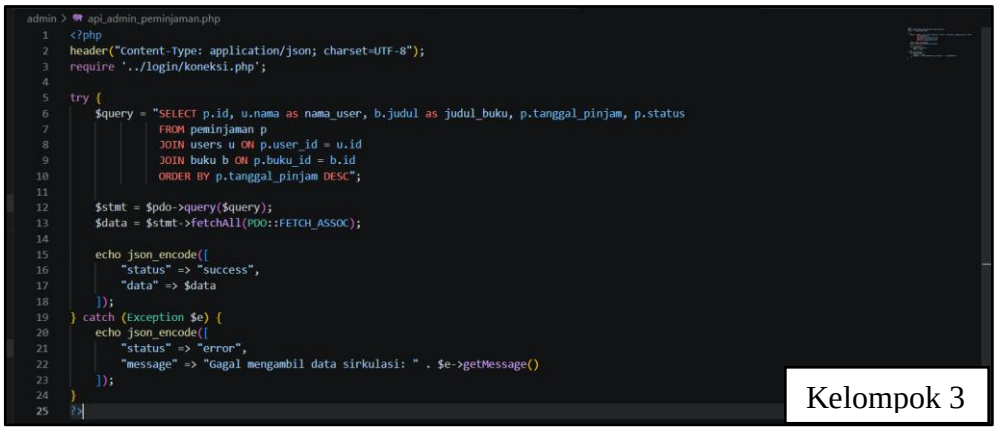


Gambar 2.39 Antarmuka Konfirmasi Perpanjangan Masa Pinjam

Penjelasan Tampilan SI:

Gambar di atas memvisualisasikan implementasi *frontend* dari fitur sirkulasi mandiri pada halaman "Rak Buku Virtual & Riwayat" yang diakses oleh anggota. Saat pengguna menekan tombol aksi "Perpanjang" pada baris data buku yang sedang dipinjam, sistem dirancang untuk tidak langsung mengeksekusi kueri ke *database*. Sebaliknya, sistem memunculkan jendela *modal* konfirmasi (*Double Confirmation*) yang sangat informatif. *Modal* ini secara transparan mengomunikasikan aturan bisnis perpustakaan (*business logic*) melalui pesan "Waktu pinjam akan ditambah 7 hari (Maks 2x perpanjangan)". Pendekatan antarmuka ini memastikan bahwa anggota sepenuhnya memahami konsekuensi dan batasan sistem sebelum memberikan konfirmasi akhir melalui tombol "Ya, Perpanjang". Aksi konfirmasi inilah yang kemudian memicu pengiriman permintaan (*request*) berformat JSON ke modul *backend* api_perpanjang.php. Selain itu, pada antarmuka tabel di latar belakang, terlihat pula ketersediaan tombol "Beri Ulasan" untuk transaksi yang telah selesai, membuktikan bahwa alur navigasi dari sisi pengguna telah dirancang secara komprehensif dan saling terintegrasi.

Printscreen source code:



Gambar 2.40 Source Code Logika Relasi Data Sirkulasi (api_admin_peminjaman.php)

Source code:

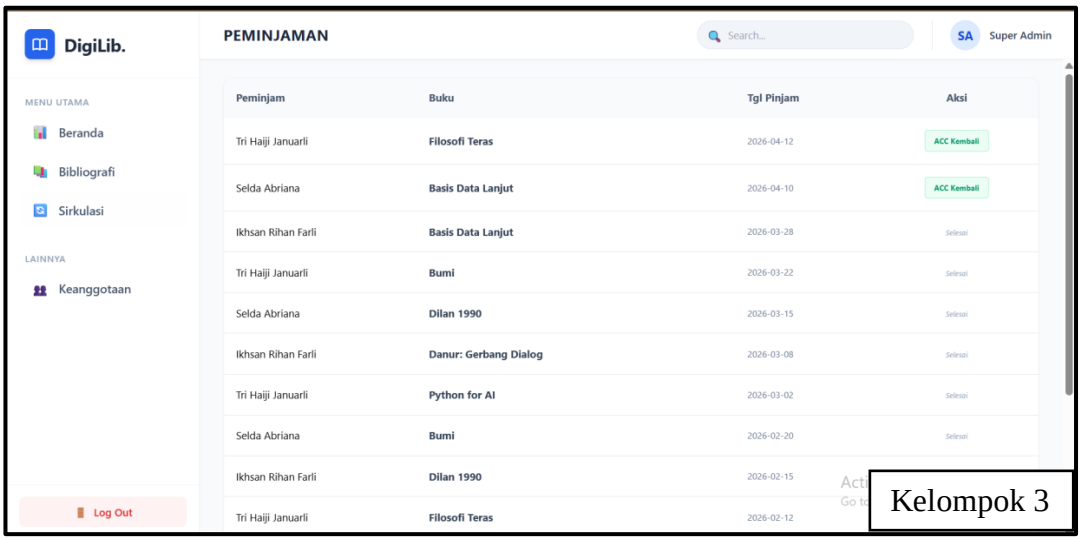
```
<?php
header("Content-Type: application/json; charset=UTF-8");
require '../login/koneksi.php';

try {
   $query = "SELECT p.id, u.nama as nama_user, b.judul as judul_buku, p.tanggal_pinjam, p.status
            FROM peminjaman p
            JOIN users u ON p.user_id = u.id
            JOIN buku b ON p.buku_id = b.id
            ORDER BY p.tanggal_pinjam DESC";

   $stmt = $pdo->query($query);
   $data = $stmt->fetchAll(PDO::FETCH_ASSOC);

   echo json_encode([
      "status" => "success",
      "data" => $data
   ]);
} catch (Exception $e) {
   echo json_encode([
      "status" => "error",
      "message" => "Gagal mengambil data sirkulasi: " . $e->getMessage()
   ]);
}
?>}
```

Pembahasan Source Code :

Modul api_admin_peminjaman.php bertugas untuk menyajikan rekapitulasi riwayat sirkulasi perpustakaan secara komprehensif bagi administrator. Mengingat tabel peminjaman hanya menyimpan kunci tamu (*foreign key*), skrip ini mengimplementasikan kueri relasional tingkat lanjut menggunakan klausa *Inner Join* yaitu SELECT p.id, u.nama as nama_user, b.judul as judul_buku, p.tanggal_pinjam, p.status FROM peminjaman p JOIN users u ON p.user_id = u.id JOIN buku b ON p.buku_id = b.id ORDER BY p.tanggal_pinjam DESC. Kueri tersebut secara dinamis menggabungkan

entitas pengguna dan buku untuk mendapatkan representasi nama asli dan judul secara utuh. Hasil ekstraksi data kemudian dikonversi menggunakan fungsi $stmt->fetchAll(PDO::FETCH_ASSOC) dan dikembalikan ke klien dalam bentuk objek JSON melalui perintah echo json_encode(["status" => "success", "data" => $data]). Pada sisi *frontend*, respons JSON tersebut diurai (*parsing*) dan diproyeksikan ke dalam antarmuka tabel "Master Data Peminjaman" secara *real-time*. Melalui antarmuka ini, administrator dapat dengan mudah memantau status sirkulasi seperti koleksi yang sedang dipinjam maupun yang telah dikembalikan—berdasarkan kronologi waktu peminjaman terbaru, tanpa harus berhadapan dengan data ID numerik yang membingungkan.

Tampilan SI:

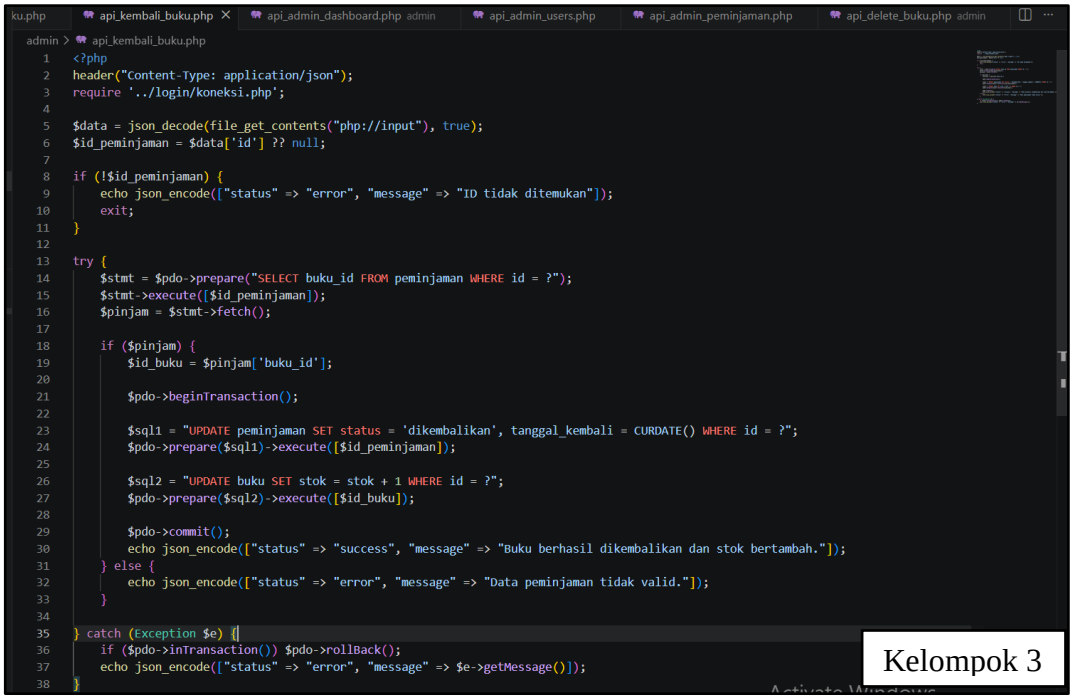


Gambar 2.41 Antarmuka Katalog Koleksi Buku (Sisi Admin)

Penjelasan Tampilan SI:

Halaman Sirkulasi pada sisi Admin dirancang khusus sebagai pusat kendali untuk mengelola seluruh transaksi peminjaman buku secara terpusat. Fokus utama antarmuka ini terletak pada tabel data peminjaman yang menyajikan informasi detail mengenai nama peminjam, judul buku, serta tanggal pinjam secara kronologis berdasarkan urutan terbaru. Administrator diberikan kendali penuh melalui kolom aksi yang dinamis; di mana tersedia tombol 'ACC Kembali' berwarna hijau yang berfungsi sebagai pemicu (*trigger*) untuk memvalidasi pengembalian fisik buku oleh anggota. Setelah administrator menekan tombol tersebut, sistem akan memproses pembaruan data ke basis data dan secara otomatis mengubah status aksi menjadi label 'Selesai' berwarna abu-abu sebagai penanda bahwa transaksi telah rampung dan stok buku telah kembali normal. Keberadaan fitur pencarian (*Search Bar*) di bagian atas semakin mempermudah administrator dalam memfilter data transaksi spesifik secara cepat, sehingga seluruh alur sirkulasi pada sistem DigiLib dapat terpantau dengan sangat akurat, transparan, dan efisien.

Printscreen source code:



Gambar 2.42 Source Code Logika Pengembalian Buku (api_kembali_buku.php)

Source code:

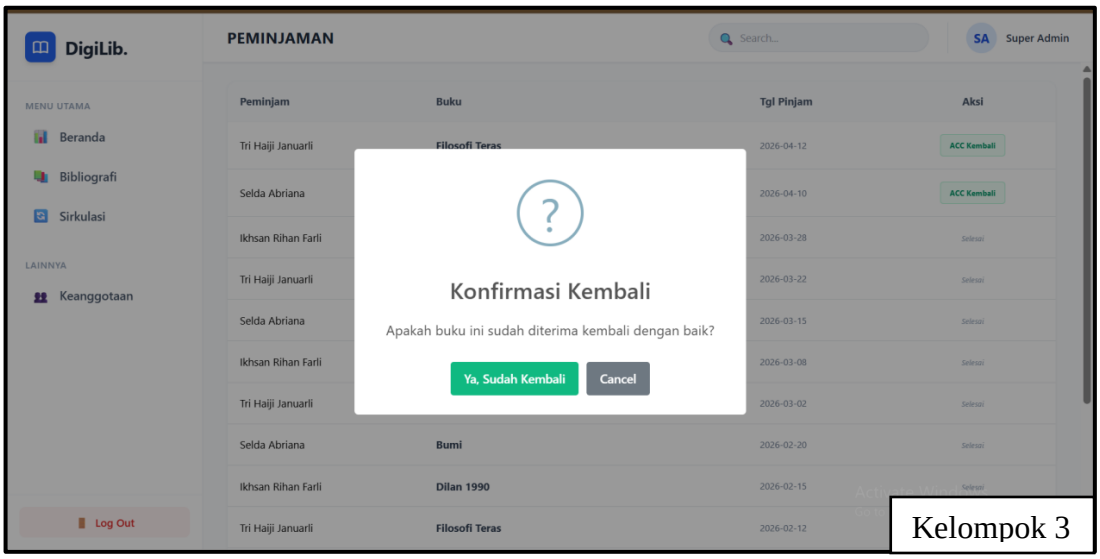
```
<?php
header("Content-Type: application/json");
require '../login/koneksi.php';

$data = json_decode(file_get_contents("php://input"), true);
$id_peminjaman = $data['id'] ?? null;

if (!$id_peminjaman) {
    echo json_encode(["status" => "error", "message" => "ID tidak ditemukan"]);
    exit;
}

try {
    $stmt = $pdo->prepare("SELECT buku_id FROM peminjaman WHERE id = ?");
    $stmt->execute([$id_peminjaman]);
    $pinjam = $stmt->fetch();

    if ($pinjam) {
        $id_buku = $pinjam['buku_id'];
```

```
        $pdo->beginTransaction();

        $sql1 = "UPDATE peminjaman SET status = 'dikembalikan', tanggal_kembali = CURDATE() WHERE id = ?";
        $pdo->prepare($sql1)->execute([$id_peminjaman]);

        $sql2 = "UPDATE buku SET stok = stok + 1 WHERE id = ?";
        $pdo->prepare($sql2)->execute([$id_buku]);

        $pdo->commit();
        echo json_encode(["status" => "success", "message" => "Buku berhasil dikembalikan dan stok bertambah."]);
    } else {
        echo json_encode(["status" => "error", "message" => "Data peminjaman tidak valid."]);
    }

} catch (Exception $e) {
    if ($pdo->inTransaction()) $pdo->rollBack();
    echo json_encode(["status" => "error", "message" => $e->getMessage()]);
}
```

Pembahasan Source Code :

Modul api_kembali_buku.php merupakan komponen kritikal dalam sistem sirkulasi yang menangani proses pengembalian buku sekaligus pemulihan jumlah stok secara otomatis. Skrip ini menangkap input ID peminjaman secara dinamis melalui fungsi $data = json_decode(file_get_contents("php://input"), true). Keunggulan utama dari logika ini adalah penerapan sistem kontrol transaksi basis data (*Database Transaction*) untuk menjamin konsistensi modifikasi data yang melibatkan lebih dari satu entitas. Proses manipulasi diawali dengan deklarasi $pdo->beginTransaction(), yang kemudian dilanjutkan dengan eksekusi kueri DML pertama yaitu UPDATE peminjaman SET status = 'dikembalikan', tanggal_kembali = CURDATE() WHERE id = ? guna menutup riwayat peminjaman dengan cap waktu (*timestamp*) hari ini. Setelah itu, sistem mengeksekusi kueri kedua berupa UPDATE buku SET stok = stok + 1 WHERE id = ? untuk memperbarui ketersediaan fisik buku di dalam katalog. Apabila kedua instruksi tersebut

berhasil dijalankan tanpa kendala, sistem akan menyegel perubahan tersebut ke dalam basis data secara permanen menggunakan perintah $pdo->commit() dan merespons klien dengan echo json_encode(["status" => "success", "message" => "Buku berhasil dikembalikan dan stok bertambah."]). Sebaliknya, apabila terjadi anomali sistem di tengah proses, blok penanganan eksepsi akan memicu perintah $pdo->rollBack() untuk membatalkan seluruh rentetan perubahan secara serentak, sehingga mencegah terjadinya desinkronisasi antara status pinjaman dan riil stok buku. Pada antarmuka pengguna, instruksi kompleks ini diinisiasi cukup dengan menekan tombol aksi pada baris data terkait di tabel sirkulasi, yang kemudian memicu pembaruan status antarmuka secara *real-time*.

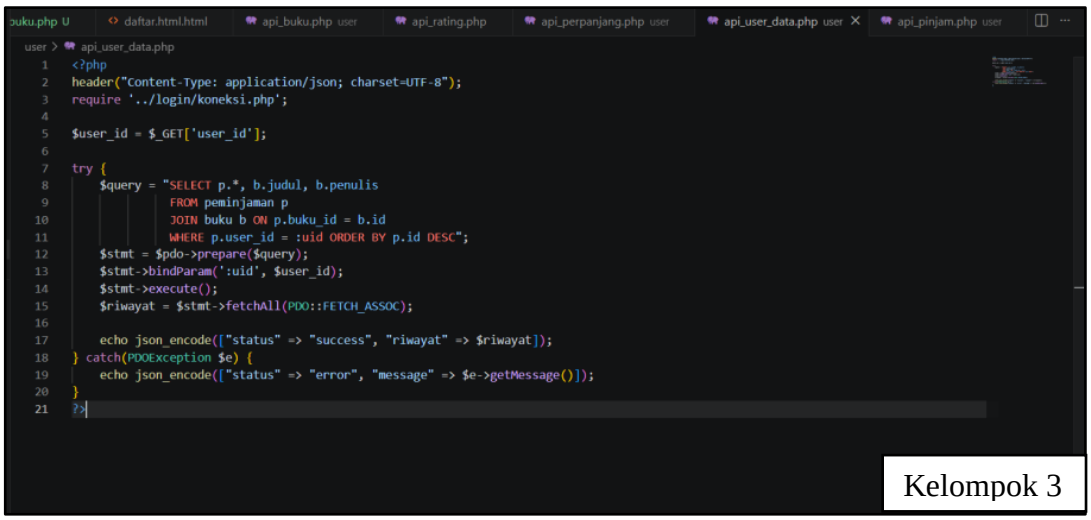


Gambar 2.43 Konfirmasi Pengembalian Buku (Sisi Admin)

Penjelasan Tampilan SI:

Rangkaian gambar di atas menampilkan representasi visual dari lapisan *frontend* yang diakses secara eksklusif oleh pengguna dengan tingkat otorisasi administrator. Pada halaman Dasbor, sistem mengamankan akses rute halaman dengan mengeksekusi validasi sesi melalui pengecekan kondisi $_SESSION['role'] === 'admin' untuk memastikan hanya staf berwenang yang dapat melihat rekapitulasi data sistem. Sementara itu, pada halaman Katalog Buku, sistem memproyeksikan seluruh data koleksi literatur perpustakaan dengan mengeksekusi perintah DQL utama yaitu SELECT * FROM buku. *Output* dari kueri basis data tersebut kemudian diurai menggunakan struktur perulangan foreach ($data as $buku) dan di-render ke dalam antarmuka berbasis *grid* (*card view*) yang sangat rapi. Tampilan ini tidak hanya menampilkan atribut visual dan tekstual seperti sampul, judul, dan sisa stok secara langsung, tetapi juga dirancang khusus untuk memberikan kemudahan bagi administrator dalam melakukan inspeksi visual (*visual monitoring*) terhadap ketersediaan aset literatur secara *real-time* sebelum melakukan operasi manajerial (CRUD) lebih lanjut.

Printscreen source code:



Gambar 2.44 Source Code Logika Riwayat Peminjaman (api_user_data.php)

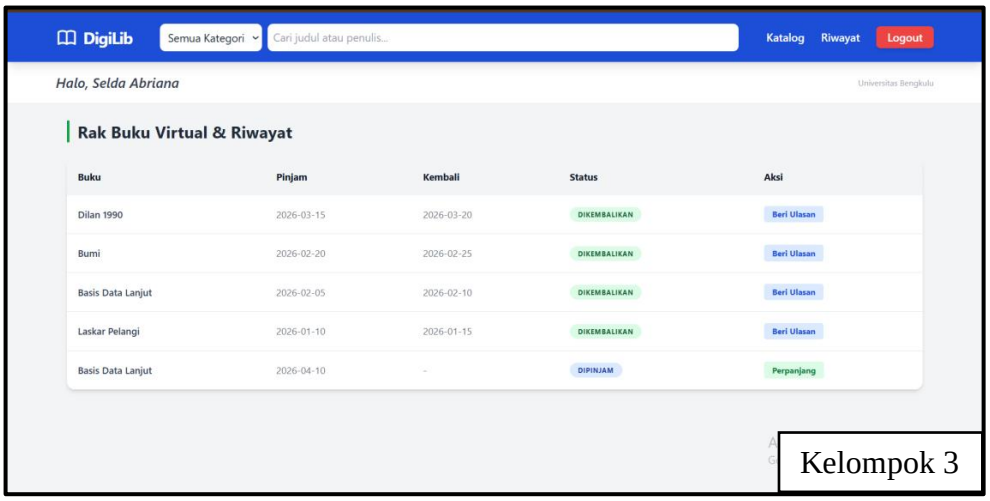
Source code:

```
<?php
header("Content-Type: application/json; charset=UTF-8");
require '../login/koneksi.php';

$user_id = $_GET['user_id'];

try {
    $query = "SELECT p.*, b.judul, b.penulis
            FROM peminjaman p
            JOIN buku b ON p.buku_id = b.id
            WHERE p.user_id = :uid ORDER BY p.id DESC";
    $stmt = $pdo->prepare($query);
    $stmt->bindParam(':uid', $user_id);
    $stmt->execute();
    $riwayat = $stmt->fetchAll(PDO::FETCH_ASSOC);

    echo json_encode(["status" => "success", "riwayat" => $riwayat]);
} catch(PDOException $e) {
    echo json_encode(["status" => "error", "message" => $e->getMessage()]);

}
?>
```

Pembahasan Source Code :

Modul api_user_data.php bertugas untuk menyajikan rekam jejak aktivitas sirkulasi secara spesifik untuk masing-masing anggota perpustakaan. Skrip ini menangkap parameter identitas pengguna melalui metode HTTP GET menggunakan instruksi $user_id = $_GET['user_id']. Untuk merelasikan data transaksi dengan detail literatur secara utuh, sistem mengeksekusi kueri DQL tingkat lanjut yang mengimplementasikan klausa *Inner Join*, yaitu SELECT p.*, b.judul, b.penulis FROM peminjaman p JOIN buku b ON p.buku_id = b.id WHERE p.user_id = :uid ORDER BY p.id DESC. Kueri ini memfilter keluaran data secara ketat menggunakan klausa WHERE p.user_id = :uid guna memastikan bahwa anggota hanya memiliki hak akses baca terhadap riwayat peminjamannya sendiri, sehingga privasi data antar pengguna tetap terjaga. Selain itu, penggunaan fungsi pengurutan ORDER BY p.id DESC memastikan bahwa riwayat transaksi yang paling baru akan selalu ditampilkan pada urutan teratas di layar antarmuka pengguna. Eksekusi kueri ini diamankan dari potensi injeksi menggunakan *prepared statements* melalui $stmt->bindParam(':uid', $user_id) sebelum himpunan data hasil ekstraksi dikonversi menjadi *array* asosiatif dengan $stmt->fetchAll(PDO::FETCH_ASSOC) dan dikirimkan kembali ke *frontend* dalam format JSON melalui perintah echo json_encode(["status" => "success", "riwayat" => $riwayat]).

Tampilan SI:



Gambar 2.45 Antarmuka Rak Buku Virtual dan Riwayat Peminjaman (Sisi User)

Penjelasan Tampilan SI:

Gambar di atas merupakan representasi visual dari halaman "Rak Buku Virtual & Riwayat" yang berfungsi sebagai dasbor sirkulasi personal bagi anggota. Antarmuka ini bertugas me-render paket data JSON yang didistribusikan oleh modul *backend* api_user_data.php. Sistem mengurai (*parsing*) *array* riwayat sirkulasi ke dalam bentuk tabel yang terstruktur, menampilkan informasi esensial seperti Judul Buku, Tanggal

Pinjam, Tanggal Kembali, serta indikator Status. Keunggulan arsitektural pada tampilan ini terletak pada implementasi *conditional rendering* (perenderan bersyarat) pada kolom Aksi. Sistem secara cerdas membaca nilai status dari basis data; apabila status sirkulasi tercatat "DIKEMBALIKAN", maka komponen yang diaktifkan adalah tombol "Beri Ulasan" yang akan memicu *modal* penilaian. Sebaliknya, jika baris data menunjukkan status "DIPINJAM".

Printscreen source code:

```
45 • CREATE TABLE ulasan (
46       id INT AUTO_INCREMENT PRIMARY KEY,
47       user_id INT,
48       buku_id INT,
49       rating INT CHECK (rating BETWEEN 1 AND 5),
50       komentar TEXT,
51       FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
52       FOREIGN KEY (buku_id) REFERENCES buku(id) ON DELETE CASCADE
53   );
54
55   ---
56   --- SEEDING DATA DUMMY
57   ---
```

Kelompok 3

Gambar 2.46 Membuat Tabel Ulasan

Source code:

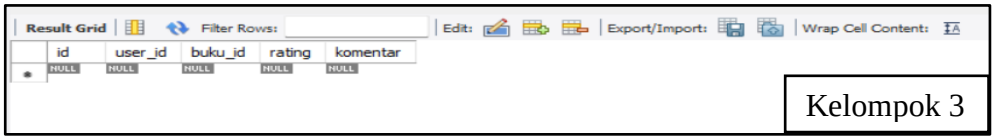
```
CREATE TABLE ulasan (
   id INT AUTO_INCREMENT PRIMARY KEY,
   user_id INT,
   buku_id INT,
   rating INT CHECK (rating BETWEEN 1 AND 5),
   komentar TEXT,
   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
   FOREIGN KEY (buku_id) REFERENCES buku(id) ON DELETE CASCADE
);


---
--- SEEDING DATA DUMMY
---
```

Pembahasan Source Code :

Perintah CREATE TABLE ulasan digunakan untuk membuat tabel ulasan yang berfungsi menyimpan data penilaian dan komentar pengguna terhadap buku. Kolom id bertipe INT dengan AUTO_INCREMENT dan dijadikan PRIMARY KEY, sehingga setiap ulasan memiliki identitas unik yang dibuat otomatis. Kolom user_id dan buku_id

bertipe INT berfungsi sebagai foreign key yang menghubungkan tabel ulasan dengan tabel users dan buku, sehingga dapat diketahui siapa yang memberikan ulasan dan untuk buku apa. Kolom rating bertipe INT dilengkapi dengan CHECK (rating BETWEEN 1 AND 5) yang membatasi nilai hanya dari 1 sampai 5. Kolom komentar bertipe TEXT digunakan untuk menyimpan isi ulasan dalam bentuk teks. Selain itu, terdapat constraint FOREIGN KEY dengan aturan ON DELETE CASCADE, yang berarti jika data user atau buku dihapus, maka ulasan yang terkait juga akan ikut terhapus. Bagian komentar SEEDING DATA DUMMY menunjukkan bahwa setelah pembuatan tabel ini biasanya akan diisi dengan data contoh untuk keperluan pengujian.

Printscreen Output:

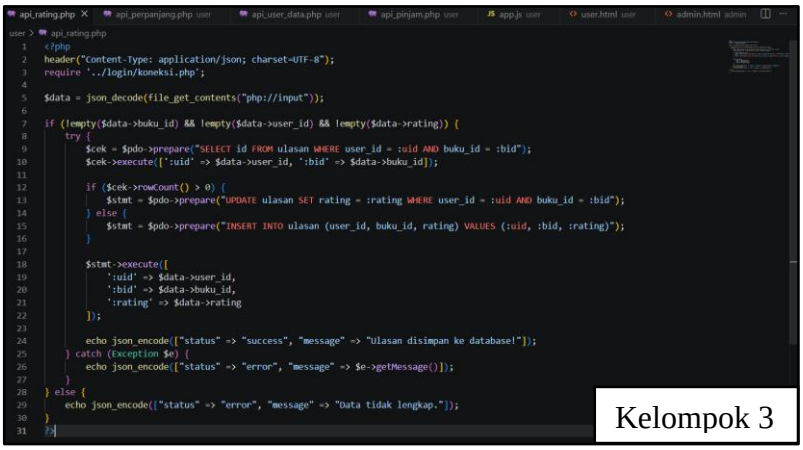


Gambar 2.47 Tabel Ulasan

Penjelasan Output:

Gambar di atas menampilkan struktur tabel ulasan pada database perpustakaan_digital. Tabel ini berfungsi untuk menampung data umpan balik atau testimoni dari para anggota mengenai koleksi buku yang telah mereka baca. Dalam tabel ini, terdapat kolom id sebagai identitas unik catatan ulasan, serta kolom user_id dan buku_id yang berfungsi sebagai kunci tamu untuk mengaitkan ulasan tersebut dengan pengguna dan buku yang bersangkutan. Tabel ini juga memiliki kolom rating untuk menyimpan penilaian skor (biasanya skala 1 hingga 5) dan kolom komentar yang berisi teks ulasan dari pengguna. Peran tabel ulasan sangat penting dalam sistem informasi ini karena memberikan nilai tambah berupa interaksi antar pengguna dan menjadi bahan pertimbangan bagi anggota lain sebelum memilih buku untuk dipinjam.

Printscreen source code:



Gambar 2.48 Source Code Pemberian Ulasan Buku (api_rating.php)

Source code:

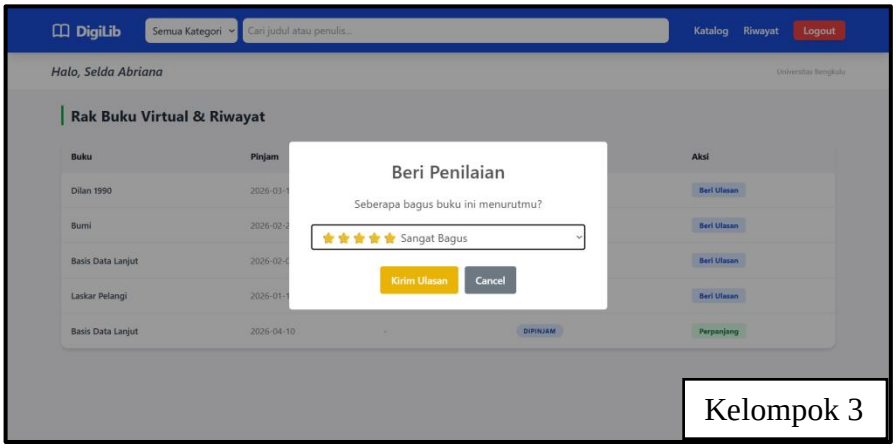
```
<?php
header("Content-Type: application/json; charset=UTF-8");
require '../login/koneksi.php';

$data = json_decode(file_get_contents("php://input"));

if (!empty($data->buku_id) && !empty($data->user_id) && !empty($data->rating)) {
    try {
        $cek = $pdo->prepare("SELECT id FROM ulasan WHERE user_id = :uid AND
buku_id = :bid");
        $cek->execute[':uid' => $data->user_id, ':bid' => $data->buku_id]);

        if ($cek->rowCount() > 0) {
            $stmt = $pdo->prepare("UPDATE ulasan SET rating = :rating WHERE user_id =
:uid AND buku_id = :bid");
        } else {
            $stmt = $pdo->prepare("INSERT INTO ulasan (user_id, buku_id, rating) VALUES
(:uid, :bid, :rating)");
        }

        $stmt->execute([
            ':uid' => $data->user_id,
            ':bid' => $data->buku_id,
            ':rating' => $data->rating
        ]);

        echo json_encode(["status" => "success", "message" => "Ulasan disimpan ke
database!"]);
    } catch (Exception $e) {
        echo json_encode(["status" => "error", "message" => $e->getMessage()]);
    }
} else {
    echo json_encode(["status" => "error", "message" => "Data tidak lengkap."]);
}
```

?>

Pembahasan Source Code :

Modul api_rating.php berfungsi untuk mengelola interaksi pemberian nilai dari anggota terhadap koleksi buku secara dinamis menggunakan format pertukaran data JSON. Skrip diawali dengan memanggil konfigurasi database melalui perintah require '../login/koneksi.php'; dan menangkap kiriman data dari *frontend* menggunakan instruksi json_decode(file_get_contents("php://input")). Keunggulan utama dari logika pada file ini adalah adanya pengecekan duplikasi ulasan melalui kueri DQL (*Data Query Language*) yaitu SELECT id FROM ulasan WHERE user_id = :uid AND buku_id = :bid. Jika sistem mendeteksi bahwa pengguna tersebut sudah pernah memberikan penilaian pada buku yang sama melalui fungsi $cek->rowCount() > 0, maka program secara otomatis akan beralih ke kueri pembaruan data berupa UPDATE ulasan SET rating = :rating.... Sebaliknya, jika data ulasan belum ditemukan, sistem akan mengeksekusi kueri DML INSERT INTO ulasan (user_id, buku_id, rating) VALUES (:uid, :bid, :rating). Seluruh proses ini diimplementasikan menggunakan teknik *prepared statements* pada objek PDO dan diakhiri dengan pengiriman respon balik ke aplikasi menggunakan echo json_encode(...) untuk memberikan notifikasi status penyimpanan ulasan kepada pengguna.

Tampilan SI:



Gambar 2.49 Antarmuka Formulir Pemberian Ulasan (Sisi User)

Penjelasan Tampilan SI:

Gambar di atas menampilkan antarmuka *modal* "Beri Penilaian" yang memfasilitasi anggota untuk memberikan umpan balik pada buku yang telah dikembalikan. Pengguna memberikan penilaian melalui komponen interaktif lima bintang, yang sekaligus berfungsi sebagai validasi sisi klien (*client-side validation*) agar input sesuai dengan *constraint* basis data (nilai 1-5). Saat tombol "Kirim Ulasan" ditekan, parameter data dibungkus dalam format JSON dan ditransmisikan secara asinkron ke modul api_rating.php untuk diproses melalui kueri INSERT ataupun UPDATE pada tabel ulasan.

BAB III
PENUTUP

A. Kesimpulan

Berdasarkan proses pembuatan dan pengembangan website Sistem Informasi Perpustakaan Digital (DigiLib), dapat disimpulkan bahwa integrasi antara teknologi *frontend*, *backend*, dan basis data merupakan kunci utama keberhasilan sistem. Penggunaan HTML5 dan CSS3 telah berhasil membangun struktur serta antarmuka yang responsif dan estetik, sementara JavaScript meningkatkan interaktivitas pengguna melalui fitur pencarian dan modal bantuan yang dinamis. Di sisi *backend*, bahasa pemrograman PHP berperan vital sebagai penghubung logika bisnis yang memungkinkan pengelolaan data buku, pendaftaran anggota, hingga pengolahan transaksi peminjaman secara otomatis.

Selain itu, implementasi Basis Data Relasional (MySQL) yang dirancang secara sistematis melalui tabel-tabel seperti users, buku, kategori, dan ulasan telah memastikan integritas dan keamanan data. Dengan struktur tabel yang tepat dan penggunaan *foreign key* yang konsisten, sistem mampu melakukan pengolahan data secara *real-time*. Hal ini memberikan efisiensi tinggi bagi admin dalam mengelola inventaris buku serta memberikan pengalaman navigasi yang lancar bagi anggota dalam mengeksplorasi katalog digital Universitas Bengkulu. Secara keseluruhan, penerapan metodologi pengembangan yang terstruktur memastikan setiap fitur aplikasi berfungsi selaras dengan kebutuhan manajemen perpustakaan modern yang serba cepat. Hal ini juga menciptakan ekosistem literasi yang lebih mudah dijangkau dan berkelanjutan bagi seluruh civitas akademika.

B. Saran

Saran dalam suatu pengembangan sistem perpustakaan digital agar sukses adalah dengan melakukan analisis kebutuhan literasi pengguna secara mendalam untuk memastikan fitur yang dibuat, seperti filter kategori dan ulasan, sesuai dengan kebutuhan anggota nantinya. Kemudian, memastikan database dioptimalkan dengan struktur tabel relasional yang efisien dan teknik indeksasi yang tepat untuk mempercepat pencarian judul buku, terutama jika jumlah koleksi sudah sangat besar. Pada keamanan sistem juga harus menjadi prioritas dengan menerapkan enkripsi pada kata sandi pengguna dan validasi input yang ketat untuk melindungi database dari potensi ancaman siber. Selain itu, memastikan sistem tetap responsif untuk diperbarui dengan fitur-fitur baru seperti unggah *e-book* di masa mendatang.

DAFTAR PUSTAKA

Fajri, T. I. (2022). Perancangan Sistem Informasi Perpustakaan Digital Gampong Panggoi Berbasis Web. *Vol. 3, No. 2 Hal 37-43*.

Isa, J., & Thomas, A. (2022). Sistem Informasi Perpustakaan Digital Berbasis Web pada STMIK Catur Sakti Kendari. *Jurnal Sistem Informasi dan Teknik Komputer, 7*(2).

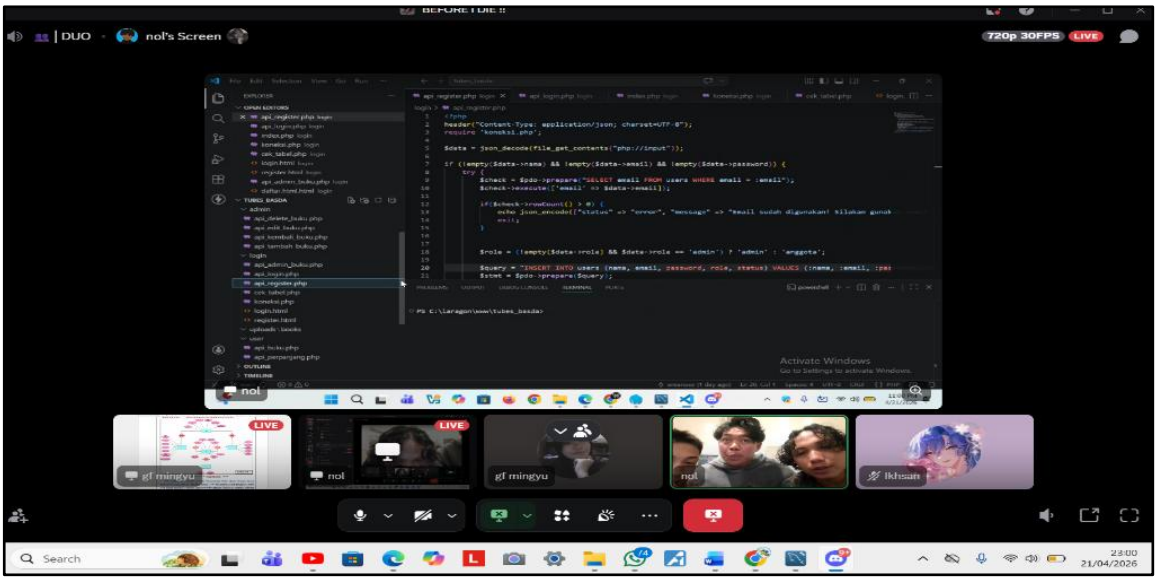
Lukmana, H. H., Alhusaini, M., & Purwayoga, V. (2023). Perancangan Sistem Informasi Perpustakaan Digital Berbasis Website Menggunakan Metode Waterfall di Jurusan Informatika Universitas Siliwangi. *METHOMIKA: Jurnal Manajemen Informatika & Komputerisasi Akuntansi, 7*(2), 340-346.

Sofwan, A. (2019). *Pemrograman Web dengan PHP dan MySQL*. IlmuKomputer.Com.

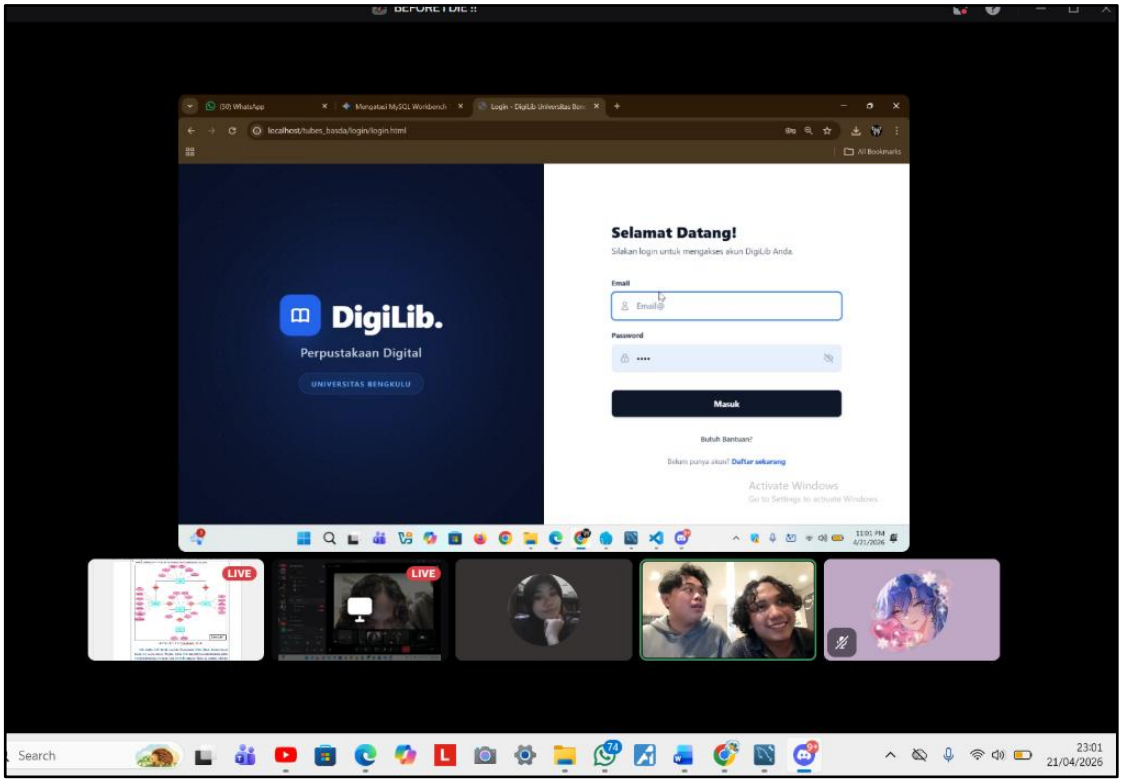
Subekti, P., & Pratama, A. (2024). Analisis dan Perancangan Sistem Informasi Perpustakaan Digital Berbasis Web. *Journal of Data Science and Information System (DIMIS), 2*(2), 70-79.

World Bank Group. (2016). *SABER-Education Management Information Systems 2016*. Education Global Practice.

JOBDESK



Lampiran 1



Lampiran 2

Keterangan Job Desk :

Project : R. Noor Fikhri (G1A024075)

Laporan : Tri Haiji Januarli (G1A024069) & R. Noor Fikhri (G1A024075)

PPT : Ikhsan Rihan Farli (G1A024083)

KEMENTERIAN PENDIDIKAN, KEBUDAYAAN
RISET DAN TEKNOLOGI
UNIVERSITAS BENGKULU
FAKULTAS TEKNIK
PROGRAM STUDI INFORMATIKA
Jl. Wr. Supratman Kandang Limun, Bengkulu Bengkulu 38371 A Telp: (0736) 344087, 22105 - 227

LEMBAR ASISTENSI

PROYEK BASIS DATA LANJUT

Nama Mahasiswa	: 1. Tri Haiji Januarli	(G1A024069)
	2. R. Noor Fikhri	(G1A024075)
	3. Ikhsan Rihan Farli	(G1A024083)
Dosen	: 1. Arie Vatresia, S.T., M.T., Ph.D., IPP	
	2. Dr. Etis Sunandi, S.Si, M.Si	
Asisten Dosen	: 1. Azzahra Faranisa	(G1A023010)
	2. Diosi Putri Arlita	(G1A023012)
	3. Salla Fikriyatul Arifah	(G1A023015)
	4. Fidia Dewi Wulandari B	(G1A023040)
	5. Anisa Julianti	(G1A023052)
	6. Ghazi Al-Ghifari	(G1A023053)
	7. Ayu Dewanti Suci	(G1A023057)
	8. Dinda Krisnauli Pakpahan	(G1A023076)
	9. Khaylillah Shafaraly Irnanda	(G1A023079)
	10. Muhammad Ryan Al Habsy	(G1A023093)

Laporan Praktikum	Catatan dan Tanda Tangan
Laporan Tugas Besar	